

ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej

Web System Programming

W06 & W07 – JavaScript, overview of
language capabilities in relation to
HTML/CSS

Syllabus

- JavaScript (JS) – scripting language
- Tag `<script>`
- JS Console
- Selected JS Elements:
 - Variables
 - Declarations `var, let, const`
 - Types/Values
 - Arrays
 - Functions and arguments
 - Objects
 - Classes
 - Conversions
 - `this`
- JS in an HTML document:
 - Access HTML elements
 - DOM
 - Manipulating HTML elements
 - Manipulating CSS styles
 - Events and reactions on them

JavaScript – generally 1/2

- Similarity – syntactically to Java:
 - Loops, conditional statements, substitution operator, mathematical, relational, logical operations, etc.,
 - semicolon at the end of the statement
 - Dot operator for objects
 - Partially – methods/functions, variables
- Scripting language:
 - Syntax errors – the script will not start
 - Errors detected during execution – then the script is interrupted
- Originally mainly for browsers
 - Once inefficient
 - Nowadays compilers have been rewritten (in connection with the development of web applications, especially SPA – Single Page Application), optimized.
 - There are "built-in" global variables/methods related to HTML document support.
 - You can include methods from JavaScript in HTML elements.

JavaScript – generally 2/2

- Features:
 - No specification of variable types, parameters, return values of methods
 - At any given time, a variable has a specific type, but you can always substitute an expression of a different type, and then the type of variable changes.
 - Based on hash and number-indexed arrays
 - Specifically functional
 - Objects are basically a hash array, where the key is the name of the field.
 - You can add and remove fields from an object.
 - Classes – a secondary concept to objects, present in the specification only recently (in newer versions of this language).
 - And semantically quite different from understanding classes in object-oriented languages such as C++/Java/C#, etc.

Tag <script> 1/2

- Tag <script> is text that is JavaScript (default)
- In HTML5 it is the default language, in other versions (HTML4/XHTML) you need to add an attribute of the MIME type, i.e. `type="text/javascript"`.
 - Theoretically, there could be other scripting languages (like searching for Yeti)
 - You can also use `type="application/javascript"`
- Very old browsers did not have scripting support, so to be sure they placed after the tag opening the script and before the closing tag the HTML code creating a comment from the JavaScript code (example below).
- Of course, JavaScript code is not displayed on the web page

```
<title>Old JS</title>
</head>
<body>
  <h1>Old JS</h1>
  <script type="text/javascript">
    <!--
      // tu kod JavaScript
    -->
  </script>
  <p>Lorem ipsum dolor ...</p>
</body>
</html>
```

```
script{
  display:none
}
```

Tag `<script>` 2/2

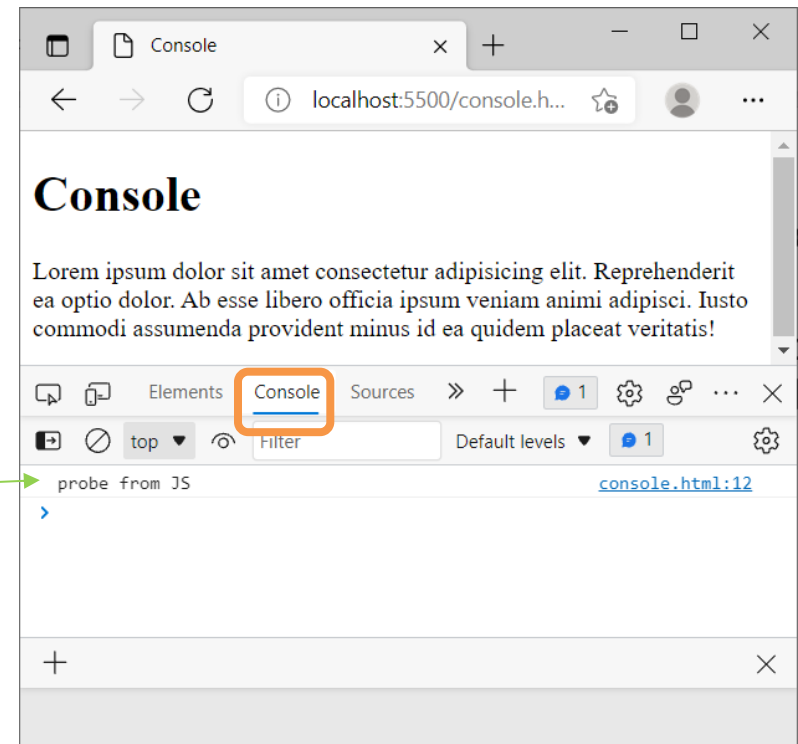
- Tag `<script>` can be used in two ways:
 - Between the opening and closing tags insert the program code in Javascript
 - Add the `src` attribute with the indication of the file with the JavaScript code

```
<script type="text/javascript" src=„script localization"> </script>
```
- The insertion location is important because the code is analyzed in order of occurrence during the processing of the web page.
- By default, most of the code is either in the element `<head>` or at the end of the page. At first, you can load framework JS files
 - Finally, those fragments that **modify** something in the **current document**.

JS console

- To observe the operation of the script, you can use the console, i.e. the `console` object and its `log()` method.
- The result of this method can be observed in the browser development tool.
- Example: `console.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" ...>
  <title>Console</title>
</head>
<body>
  <h1>Console</h1>
  <script>
    console.log("probe from JS");
  </script>
  <p>Lorem ipsum dolor ...</p>
</body>
</html>
```



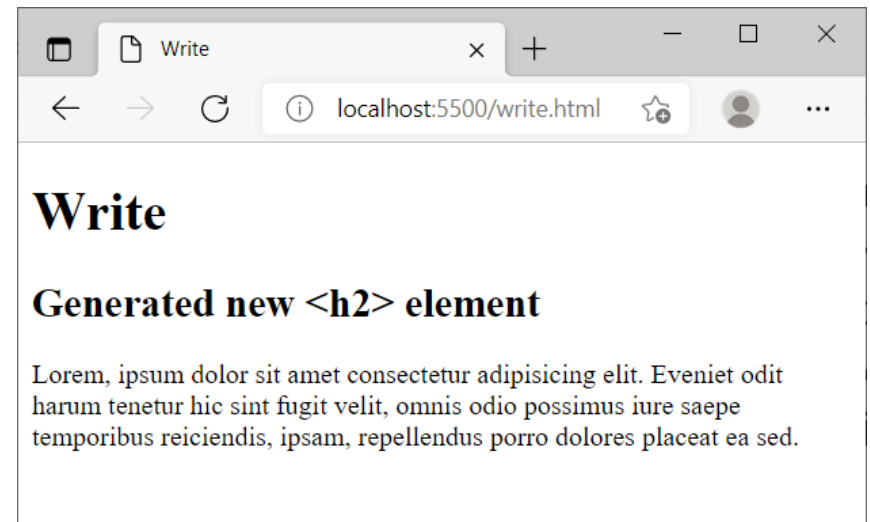
JavaScript Programs

- A computer program is a list of "instructions" to be "executed" by a computer.
- Most JavaScript programs contain many JavaScript statements. The instructions are executed one by one, **in the same order** in which they **were written**.
- In HTML, JavaScript programs are executed by a **web browser**.
- Semicolons separate JavaScript statements. There **should be a semicolon at the end** of each statement (semicolons at the end of the statement are not required, but highly recommended). When separated by semicolons, **multiple statements on a single line** are allowed.
- JavaScript ignores many spaces. You can add spaces to the script to make it easier to read.
- It is a good practice to place spaces around operators (= + - * /).
- For best readability, developers often avoid lines of code longer than **80 characters**.
- JavaScript expressions can be grouped into blocks of code, inside curly braces {...}. The purpose of creating blocks of code is to define the statements that you want to execute together.

Type text into a document

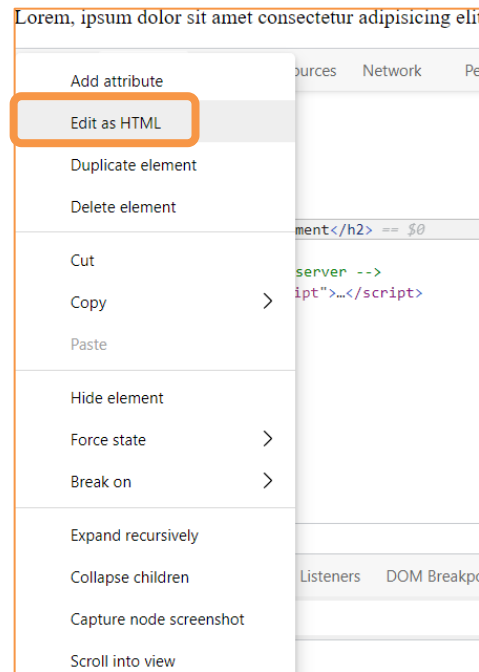
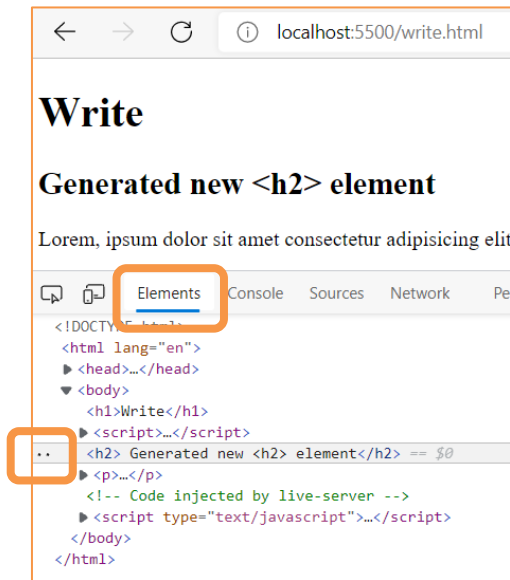
- You can use the `document` object and its `write()` and `writeln()` methods to type text into the current HTML document **where the script is executed**.
 - The second method adds a transition code to a new line at the end (but in the text of the document, this is not `<br>` so it is ignored by the browser)
- The added text becomes part of the document and is rendered.
 - That is, the document after adding new content must be correct in order for it to display correctly.
 - This text is added to the page at the current stage of rendering (that is, during the browser reading the code and its processing and display) of the page.
- Example: `write.html`

```
<title>Write</title>
</head>
<body>
  <h1>Write</h1>
  <script>
    document.writeln("<h2> Generated new");
    document.writeln("&lt;h2&gt; element</h2>");
  </script>
  <p>
    Lorem, ipsum dolor sit ...
  </p>
</body>
</html>
```



Find modified document text

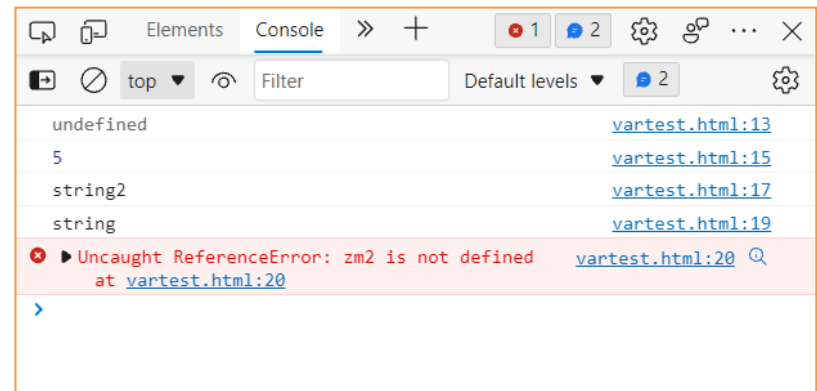
- When viewing the "page source" you will only see the text BEFORE any scripts are executed.
 - It is better to use the developer tool and the "Elements" tab
- Only after the item `<h2>` is found using the browser's development tool and using the "Edit as HTML" option for it, you can see the transition to the new line generated by `writeln()`.



Declaration with var/let/const

- Identifier of the variable (but also functions, classes, etc.)
 - String of letters, numbers, dollar characters '\$' and underscores '_'
 - Cannot begin with digit
 - case-sensitive
 - **Keywords cannot** be identifier
- Declaration of variables (precede the variable with a keyword):
 - **var** (always)
 - **let** (from ES6 2015)
 - **const** (from ES6 2015)
- A variable declared with **var** without initialization has an **undefined** value (keyword)
- With **var** you can declare a variable with the same identifier **multiple times!** It does not lose its previous value if we do not assign anything new.
- Although the type is determined by assignment to a variable, it may change the next time you assign it.
- Using an undeclared variable **breaks** the script/block of code with **exception**.
- Example: varTest.html

```
<title>var Test</title>
</head>
<body>
  <h1><code>var</code> Test</h1>
  <script>
    var zm1;           // type/value undefined
    console.log(zm1);
    zm1=3;             // type number
    console.log(zm1+2);
    zm1="string";      // type string
    console.log(zm1+2);
    var zm1;
    console.log(zm1);
    console.log(zm2);  // not exists
    console.log("after");
  </script>
  <p>
    Lorem ipsum dolor sit ...
  </p>
</body>
</html>
```



Declaration **var** issues

- They have a **global** scope – global variables!!!
 - Redeclaration in a block of code (e.g. `for` loop) operates on the same variable
- Declaring a variable with **var** within a function creates a **new, local variable**.
- Example: `varGlobal.html`

```
<title>var Global</title>
</head>
<body>
  <h1><code>var</code> Global</h1>
  <script>
    function set(){
      var zm1=20;
    }
    var zm1=5;
    {
      var zm1=10;
    }
    console.log(zm1);
    set();
    console.log(zm1);
  </script>
  <p>
    Lorem ipsum dolor sit ...
  </p>
</body>
</html>
```

10	varGlobal.html:19
10	varGlobal.html:21
>	

Visibility of the variable declared with **var**

```
<title>var From start</title>
</head>
<body>
  <h1><code>var</code> From start</h1>
  <script>
    function set(){
      lok=10; // use before declaration
      zm1=20;
      var lok; // local declaration
      return lok+zm1;
    }
    zm1=5;      // use before declaration
    var zm1;
    var x=set();
    console.log("zm1="+zm1);
    // console.log("zm2="+zm2); // generates exception
    console.log("x="+x);
    console.log("lok="+lok);
    var zm3=333; // only declaration happens
  </script>
  <p>
    Lorem ipsum dolor sit ...
  </p>
  <script>
    console.log("next fragment of script");
    var zm2=100;
    console.log("zm1="+zm1);
    console.log("zm3="+zm3);
    console.log("x="+x);
  </script>
</body>
</html>
```

- All variables declared from **var** have visibility **from the beginning** of the `<script>` element in which they occur (or from the beginning of the function), even though the declaration will be in a later piece of code (**hoisting**).

All JS codes are **one common** script, but compiled and run **in blocks** `<script>`

- Variables declared in one `<script>` element are usable in **later** script elements (but **not earlier** ones)
- Exceptions only **break** the code in the **current script element**, but all **var** declarations (without initialization) **will be** executed.

Example: `varFromStart.html`

zm1=20	varFromStart.html:21
x=30	varFromStart.html:23
✖ ▶ Uncaught ReferenceError: lok is not defined	varFromStart.html:24 🔍
at varFromStart.html:24	
next fragment of script	varFromStart.html:31
zm1=20	varFromStart.html:33
zm3=undefined	varFromStart.html:34
x=30	varFromStart.html:35
>	

Declaration of a variable using `let` and `const`

- A variable declared in a block as `let/const` **cannot** be **re-declared** in that **block**.
- In an **internal block**, such a declaration means a **new variable** whose scope **ends** at the **end of the block**.
- These variables are **visible only** from the **moment of declaration**.
- A variable declared by `let` can be changed, variables declared as `const` cannot be changed.
 - But for `const` you can still **change the content** (object, array) of such a variable, that is, it is a **constant reference** (to an array, object)
- Example: `varLetConst.html` (TODO: analyze the example)

```
<h1><code>var/let/const</code></h1>
<script>
  let a1=1;
  var zm1=1;
  const x1=1;
  const obj1={field:1};
  {
    let a1=2;
    var zm1=2;
    const x1=2;
  }
  console.log("a1="+a1+", zm1="+zm1+", x1="+x1+", obj1.field="+obj1.field);
  a1=3;
  zm1=3;
  //x1=3; Uncaught TypeError: Assignment to constant variable.
  obj1.field=3;
  console.log("a1="+a1+", zm1="+zm1+", x1="+x1+", obj1.field="+obj1.field);
</script>
<p>
  Lorem ipsum dolor sit ...
</p>
<script>
</script>
</body>
</html>
```

top	Filter	Default levels	No Issues
a1=1, zm1=2, x1=1, obj1.field=1			
a1=3, zm1=3, x1=1, obj1.field=3			
>			

Variables declared with **const**

- CAN NOT:

- Reassign a constant value
- Reassign a fixed array
- Reassign a fixed object

```
const cars = ["Saab", "Volvo", "BMW"];  
// cars = ["Toyota", "Volvo", "Audi"];
```

- But you can:

- Change the elements of a fixed array
- Change the properties of a fixed object

```
// You can create a constant array:  
const cars = ["Saab", "Volvo", "BMW"];  
// You can change an element:  
cars[0] = "Toyota";  
// You can add an element:  
cars.push("Audi");
```

Types/Values

- There are only types:
 - undefined – the variable has no value or type defined.
 - number, bigint
 - string
 - object – objects (including arrays), there is a value `null`.
 - function – functional type
 - boolean (true/false)
- Operator `typeof`
- Classes (ES6 2015) – for faster creation of objects, the name "object prototyper" or object patterns would be more appropriate.
 - Operator `new`
- Example: `types.html`

```
<script>
  let $num = 5.67;
  let str$ = "A small sentence.";
  let obj={ name:"Einstein", salary:2000};
  // existing variables
  console.log($num+", "+str$+", "+obj+", "+ obj.name+", "+obj.salary);
  console.log("types: "+(typeof $num)+", "+typeof(str$)+", "+typeof(obj));
  // undefined
  console.log("types? :"+(typeof x)+", "+typeof(obj.noExist)+", "+typeof(obj[2]));
  console.log("values? :"+ x+", "+obj.noExist+", "+obj[2]);
  // impossible
  console.log(obj.noExist.nextNotExist);
  var x=5;
</script>
```

5.67, A small sentence., [object Object], Einstein, 2000

[types.html:16](#)

types: number, string, object

[types.html:17](#)

types? :undefined, undefined, undefined

[types.html:19](#)

values? :undefined, undefined, undefined

[types.html:20](#)

✖ ▶ Uncaught TypeError: Cannot read properties of undefined (reading 'nextNotExist')
at [types.html:22](#)

Arithmetics and logical operations

- From https://www.w3schools.com/js/js_arithmetic.asp

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Remainder)
++	Increment
--	Decrement

Operator	Description
&&	and
	or
!	not

JavaScript comparison operators

- Given that `x = 5`, the table below explains the comparison operators:

Operator	Description	Comparing	Returns
==	equal to	<code>x == 8</code>	false
		<code>x == 5</code>	true
		<code>x == "5"</code>	true
===	equal value and equal type	<code>x === 5</code>	true
		<code>x === "5"</code>	false
!=	not equal	<code>x != 8</code>	true
!==	not equal value or not equal type	<code>x !== 5</code>	false
		<code>x !== "5"</code>	true
		<code>x !== 8</code>	true
>	greater than	<code>x > 8</code>	false
<	less than	<code>x < 8</code>	true
>=	greater than or equal to	<code>x >= 8</code>	false
<=	less than or equal to	<code>x <= 8</code>	true

Conversion of variable types 1/2

- The `typeof` operator returns the type of variable or expression:

```
typeof "Jan Nowak"    // returns "string"  
typeof 3.14           // returns "number"
```

- JavaScript variables can be converted to a new variable and a different data type:
 - Using the JavaScript function
 - Automatically by JavaScript alone
- The global method `Number()` can convert strings to numbers.
- The global `Number()` method can also convert logical values to numbers.
- The global method `String()` can convert numbers to strings.
- The global method `String()` can convert logical values to strings.
- The `Number.toString()` method does the same thing:
`x.toString()`
`(123).toString()`

Conversion of variable types 2/2

- The unary + operator can be used to convert a variable to a number:

```
let y = "5";           // y is a string
let x = + y;           // x is a number
```

- If the variable cannot be converted, it will still become a number, but with a NaN (Not a Number) value

```
let y = "John";        // y is a string
let x = + y;           // x is a number (NaN)
```

- Other conversions:

```
Number(false)          // returns 0
Number(true)            // returns 1
String(false)           // returns "false"
String(true)            // returns "true"
false.toString()        // returns "false"
true.toString()         // returns "true"
```

Method	Description
Number()	Returns a number, converted from its argument
parseFloat()	Parses a string and returns a floating point number
parseInt()	Parses a string and returns an integer

Date conversion

- You can use the global method `Number()` to convert dates to numbers.
`d = new Date();`
`Number(d)` // returns 1404568027739
- Date method `getTime()` does the same.
`d = new Date();`
`d.getTime()` // returns 1404568027739
- The global `String()` method can convert dates to strings.
`String(Date())` // returns "Thu Jul 17 2014
15:38:19 GMT+0200 (W. Europe Daylight Time)„
- The method `Date toString()` does the same.
`Date().toString()` // returns "Thu Jul 17 2014
15:38:19 GMT+0200 (W. Europe Daylight Time)"

Date and time methods

- https://www.w3schools.com/js/js_date_methods.asp

Method	Description
getFullYear()	Get year as a four digit number (yyyy)
getMonth()	Get month as a number (0-11)
getDate()	Get day as a number (1-31)
getDay()	Get weekday as a number (0-6)
getHours()	Get hour (0-23)
getMinutes()	Get minute (0-59)
getSeconds()	Get second (0-59)
getMilliseconds()	Get millisecond (0-999)
getTime()	Get time (milliseconds since January 1, 1970)

Automatic type conversion

- When JavaScript tries to operate on the "wrong" data type, it will try to convert the value to the "right" type.
- The result is not always what you expect:

<code>5 + null</code>	<code>// returns 5</code>	because null is converted to 0
<code>"5" + null</code>	<code>// returns "5null"</code>	because null is converted to "null"
<code>"5" + 2</code>	<code>// returns "52"</code>	because 2 is converted to "2"
<code>"5" - 2</code>	<code>// returns 3</code>	because "5" is converted to 5
<code>"5" * "2"</code>	<code>// returns 10</code>	because "5" and "2" are converted to 5 and 2
- JavaScript has dynamic types. This means that the same variable can be used to store different types of data

<code>let x;</code>	<code>// Now x is undefined</code>
<code>x = 100;</code>	<code>// Now x is a Number</code>
<code>x = "Anna";</code>	<code>// Now x is a String</code>

Conversions

Original Value	Converted to Number	Converted to String	Converted to Boolean
false	0	"false"	false
true	1	"true"	true
0	0	"0"	false
1	1	"1"	true
"0"	0	"0"	true
"000"	0	"000"	true
"1"	1	"1"	true
NaN	NaN	"NaN"	false
Infinity	Infinity	"Infinity"	true
-Infinity	-Infinity	"-Infinity"	true
""	0	""	false
"20"	20	"20"	true
"twenty"	NaN	"twenty"	true
[]	0	""	true
[20]	20	"20"	true
[10,20]	NaN	"10,20"	true
["twenty"]	NaN	"twenty"	true
["ten","twenty"]	NaN	"ten,twenty"	true
function(){}	NaN	"function(){}"	true
{ }	NaN	"[object Object]"	true
null	0	"null"	false
undefined	NaN	"undefined"	false

- Values in quotes indicate string values.
- **Red values** indicate values (some) programmers might not expect.

JavaScript bit operators

- Bit operators run on 32-bit numbers.
- Any numeric operand in the operation is converted to a 32-bit number. The result is converted back to a JavaScript number.
- Examples below for 4-bit operands.

Operator	Description	Example	Same as	Result	Decimal
&	AND	5 & 1	0101 & 0001	0001	1
	OR	5 1	0101 0001	0101	5
~	NOT	~ 5	~0101	1010	10
^	XOR	5 ^ 1	0101 ^ 0001	0100	4
<<	left shift	5 << 1	0101 << 1	1010	10
>>	right shift	5 >> 1	0101 >> 1	0010	2
>>>	unsigned right shift	5 >>> 1	0101 >>> 1	0010	2

Assignment operators and shift assignment operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Operator	Example	Same As
<<=	x <<= y	x = x << y
>>=	x >>= y	x = x >> y
>>>=	x >>>= y	x = x >>> y

Bitwise Assignment Operators and Logical Assignment Operators

Operator	Example	Same As
<code>&=</code>	<code>x &= y</code>	<code>x = x & y</code>
<code>^=</code>	<code>x ^= y</code>	<code>x = x ^ y</code>
<code> =</code>	<code>x = y</code>	<code>x = x y</code>

Operator	Example	Same As
<code>&&=</code>	<code>x &&= y</code>	<code>x = x && (x = y)</code>
<code> =</code>	<code>x = y</code>	<code>x = x (x = y)</code>
<code>??=</code>	<code>x ??= y</code>	<code>x = x ?? (x = y)</code>

Undefined variables

- In JavaScript, a variable without a value is undefined. The type of such a variable is also undefined.

```
let auto;      // Value is undefined, type is undefined
```

- Each variable can be emptied by setting its value to undefined. The type will also be undefined.

```
auto = undefined;    // Value is undefined, type is undefined
```

- An empty value has nothing to do with undefined.
- A null value is different from undefined.
- An empty string has both a valid value and a type.

```
let auto = "";      // Value is "", type is "string"
```

JavaScript strings

- A string (or text string) is a series of characters, such as "John Smith,,

- Strings are written in quotation marks. You can use single or double quotation marks:

```
let carName1 = "Toyota Corolla";    //
```

Using double quotes

```
let carName2 = 'Toyota Corolla';    //
```

Using single quotes

- You can use quotation marks inside a string as long as they don't match the quotation marks surrounding the string:

- ```
let answer1 = "It's alright"; //
```

Single quote inside double quotes

```
let answer2 = "She is called 'Anna'"; //
```

Single quotes inside double quotes

```
let answer3 = 'She is called "Joanna"'; //
```

Double quotes inside single quotes

# Numbers

- Numbers can be written as integer or decimal:  

```
let x1 = 12.00; // Saves with decimal part
let x2 = 12; // Saves without decimal part
```
- Very large or very small numbers can be written in scientific (exponential) notation:  

```
let y = 1234e5; // 123400000
let z = 1234e-5; // 0.01234
```
- Logical values can have only two values: true or false.  

```
let x = 1;
let y = 1;
let z = 2;
(x == y) // returns true
(x == z) // returns false
```

## Arrays indexed by numbers

- Arrays are special objects with many functions and a length field, indexed with non-negative integers
  - Attention! theoretically, you can also add `negative`, non-integer and non-numeric indices – because this is still correct **for the dictionary**. Theoretically, an index can be another object, function, or array. The `length` field remembers **the largest positive** integer index.
- Access items using indexed square brackets.
- Arrays in JavaScript are written in square brackets and the individual elements of the array are separated by commas.
- The indexes of the array are counted from zero, which means that the first element is [0], the second is [1], and so on.  
`const cars = ["Toyota", "Lexus", "Fiat"];`
- Can be created by writing a curly notation or constructor for the `Array` class.
  - With **one integer** argument – the parameter specifies **the length** of the array (with elements equal to `undefined`)
  - With zero or multiple arguments: Arguments become consecutive elements of an array.

# Example of an array indexed with numbers

```
var arr=[5,10,15];
console.log("len="+arr.length+", "+arr[1]); //3, 10
arr[3]=20;
console.log("len="+arr.length+", "+arr[3]); //3, 20
console.log(arr[5]); //undefined
arr[10]=100;
console.log("len="+arr.length+", "+arr[10]); //11, 100
arr[-1]=-10;
console.log("len="+arr.length+", "+arr[-1]); //11, -10
const arr2=[]; // constant reference
arr2[2]="two";
console.log("len="+arr2.length+", "+arr2[2]); //3, two
const arr3=new Array(); //empty array
const arr4=new Array("one","two","three"); // 3-elements array
const arr5=new Array(1,"two",arr4); // 3-elements array
const arr6=new Array("one"); // 1-elements array
const arr7=new Array(11); // 11-elements array
console.log(arr3,arr4,arr5,arr6,arr7);
```

- View in JS console:

len=3, 10 [arrays.html:13](#)  
len=4, 20 [arrays.html:15](#)  
undefined [arrays.html:16](#)  
len=11, 100 [arrays.html:18](#)  
len=11, -10 [arrays.html:20](#)  
len=3, two [arrays.html:23](#)  
[arrays.html:29](#)

▼ [] ⓘ  
length: 0  
▶ [[Prototype]]: Array(0)

▼ (3) ['one', 'two', 'three'] ⓘ  
0: "one"  
1: "two"  
2: "three"  
length: 3  
▶ [[Prototype]]: Array(0)

▼ (3) [1, 'two', Array(3)] ⓘ  
0: 1  
1: "two"  
2: (3) ['one', 'two', 'three']  
length: 3  
▶ [[Prototype]]: Array(0)

▼ ['one'] ⓘ  
0: "one"  
length: 1  
▶ [[Prototype]]: Array(0)

▼ (11) [empty × 11] ⓘ  
length: 11  
▶ [[Prototype]]: Array(0)

# Objects 1/5

- In JavaScript, almost "everything" is an object.
- All JavaScript values, except the primitive values, are objects.
- A primitive value is a value that has no properties or methods.
  - 3.14 is the primitive value
- The primitive data type is data that has a primitive value.
- JavaScript defines 7 types of primitive data types:
  - `string`
  - `number`
  - `boolean`
  - `null`
  - `undefined`
  - `symbol`
  - `bigint`

## Objects 2/5

- A JavaScript object is a collection of named values.
  - Implemented as a hash table.
- JavaScript objects are containers for named values, called properties/fields and methods.
  - Name of field is a **valid** identifier
- It is common practice to declare objects with the keyword `const`.
- There are different ways to create new objects:
- Simple creation with object literal:
  - By using an object literal, you both **define** and **create** an object in a single statement.
- An object literal is a list of `name : value` pairs (e.g. `age:50`) in curly braces `{ }` separated by commas `,`.

```
const person = {firstName:"Anna",
lastName:"Jasna", age:50, eyeColor:"blue"};
```
- You can also create an empty object and then assign properties:

```
const person = {};
person.firstName = "Anna";
person.lastName = "Jasna";
person.age = 50;
person.eyeColor = "blue";
```

## Objects 3/5

- Creating object of class `Object`:
  - Create a single object with the keyword `new`. First, you create an empty object:

```
const person = new Object();
person.firstName = "Anna";
person.lastName = "Jasna";
person.age = 50;
person.eyeColor = "blue";
```
- Define an object constructor, and then create objects of the constructed type (will be explained when defining classes).
- Create an object with `Object.create()`.
- Objects are variable: they are addressed by reference, not by value.
- If the `person` is an object, the following assignment will not create a copy of the `person`:

```
const x = person; // a copy of a person is not created here.
```

- Both `x` and `person` are the same object.
- Any changes to `x` will also change the `person`, because `x` and the `person` are the same object.

## Objects 4/5

- Properties are values associated with a JavaScript object.
- A JavaScript object is a collection of unordered properties.
- Properties can usually be changed, added, and deleted, but some are read-only.
- The object property access syntax is the dot operator:  
*objectName.property*
  - `person.firstname + " is " + person.age + " years old.";`
- Or square brackets with string:  
*objectName["property"]*
  - `person["firstname"] + " is " + person["age"] + " years old.";`
- In square brackets can be also an expression:  
*objectName[expression]*
  - `x = "age";  
person[x] ;`
- *expression* is the name of the property.

# Objects 5/5

- Example of use

```
let person={ name:"Einstein", salary:2000};
console.log(person);
person.salary+=100;
console.log(person);
person.year=1900;
console.log(person);
person.name=undefined;
console.log(person);
delete person.salary;
console.log(person);
person["forname"]="Albert";
console.log(person);
person.name={() => "Name"};
console.log(person);
console.log(person.name());
```

|                                                    |                                 |
|----------------------------------------------------|---------------------------------|
| ▶ {name: 'Einstein', salary: 2000}                 | <a href="#">objects.html:13</a> |
| ▶ {name: 'Einstein', salary: 2100}                 | <a href="#">objects.html:15</a> |
| ▶ {name: 'Einstein', salary: 2100, year: 1900}     | <a href="#">objects.html:17</a> |
| ▶ {name: undefined, salary: 2100, year: 1900}      | <a href="#">objects.html:19</a> |
| ▶ {name: undefined, year: 1900}                    | <a href="#">objects.html:21</a> |
| ▶ {name: undefined, year: 1900, forname: 'Albert'} | <a href="#">objects.html:23</a> |
| ▶ {year: 1900, forname: 'Albert', name: f}         | <a href="#">objects.html:25</a> |
| Name                                               | <a href="#">objects.html:26</a> |

## Liskov substitution rule:

Functions that use pointers or references to base classes must also be able to use objects that inherit from base classes, without knowing these objects exactly.

# Objects – different ways to access the object

```
<!DOCTYPE html>
<html>
 <!-- file: object.html-->
 <head>
 <title>Object</title>
 </head>
 <body>

 <h2>JavaScript Objects</h2>
 <p id="demo"></p>

 <script>
const myObj = {
 name: "Anna",
 age: 40,
 cars: {
 car1: "Ford",
 car2: "BMW",
 car3: "Toyota"
 }
}
//document.getElementById("demo").innerHTML = myObj.cars.car2;
//document.getElementById("demo").innerHTML = myObj.cars["car2"];
document.getElementById("demo").innerHTML = myObj["cars"]["car2"];
</script>

 </body>
</html>
```



## Comments

- JavaScript comments can be used to explain JavaScript code and make it more readable.
- JavaScript comments can also be used to prevent execution when testing alternative code or individual statements.
- Single-line comments begin with `//`. Any text between `//` and the end of the line will be ignored by JavaScript (will not be executed).
- Multiline comments begin with `/*` and end with `*/`. Any text between `/*` and `*/` will be ignored by JavaScript.  

```
let x = 5; // Declaration x,
 assign a value of 5 there
let y = x + 2; // Declaration y,
 assign a value of 2 there
```
- Most often, one-line comments are used. Block comments are often used in formal documentation.

# Functions

- Functions can be declared in the following two ways
  - The first way is just a notation shortcut.
  - You produce a variable called a function, which is of type function.
  - Like any variable, you can change its value and/or type.
- Function parameters do not have specified types
  - May have default values
- The result type may depend on the execution of the function
- **Arguments** not specified in the function call are `undefined`.
- **Redundant** arguments are **ignored**.
  - Not quite (further slides)
- From ES6, the second way of defining the so-called anonymous function can be used, using the **lambda expression**, e.g..

```
var addNew=(x,y) => x+y;
```

or

```
var addNew=(x,y) => { return x+y;;}
```

```
function add(x,y){
 let result=x+y;
 return result;
}
console.log(add(2,3));
var addNew=function(x,y){ // var/let/const
 return x+y;
}
console.log(addNew(2,3));
add=function(name){ return "name="+name;};
console.log(add("Jane"));
console.log(add(2,3));
console.log(add());
```

5	<a href="#">functions.html:16</a>
5	<a href="#">functions.html:20</a>
name=Jane	<a href="#">functions.html:22</a>
name=2	<a href="#">functions.html:23</a>
name=undefined	<a href="#">functions.html:24</a>

# Function parameters

- There can **only** be **one** function with a **given name**.
  - No function overloading
- You can create a long list of parameters and check which are defined
- You can use an array of parameters because for each function call, an array indexed from 0 is created with the arguments of the function call under the local variable `arguments`, which is similar to an array indexed with numbers.
- Arguments are copies of numbers, strings, or (object-check references).

```
function showArgs(x,y){
 for(let i=0;i<arguments.length;i++)
 console.log(`[${i}]=`+arguments[i]);
}
function check(){
 showArgs("Jan",6);
}
console.log("check()");
check();
console.log("showArgs()");
showArgs(3,"Jane");
console.log("showArgs() with 5 args");
showArgs(0,"Jane",2,3,4);
console.log("showArgs() with 1 arg");
showArgs("one");
```

check()	<a href="#">arguments.html:19</a>
[0]=Jan	<a href="#">arguments.html:14</a>
[1]=6	<a href="#">arguments.html:14</a>
showArgs()	<a href="#">arguments.html:21</a>
[0]=3	<a href="#">arguments.html:14</a>
[1]=Jane	<a href="#">arguments.html:14</a>
showArgs() with 5 args	<a href="#">arguments.html:23</a>
[0]=0	<a href="#">arguments.html:14</a>
[1]=Jane	<a href="#">arguments.html:14</a>
[2]=2	<a href="#">arguments.html:14</a>
[3]=3	<a href="#">arguments.html:14</a>
[4]=4	<a href="#">arguments.html:14</a>
showArgs() with 1 arg	<a href="#">arguments.html:25</a>
[0]=one	<a href="#">arguments.html:14</a>

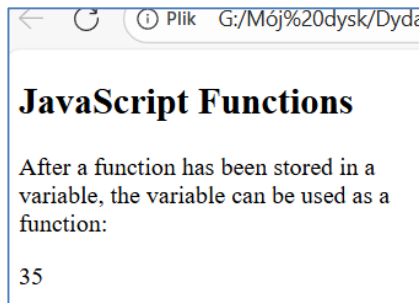
# Functions

- A JavaScript `function` is a block of JavaScript code that can be executed when it is "called".
- For example, a function can be called when an event occurs, such as when a user clicks a button.
- Place scripts at the bottom of an item `<body>` improves display speed because script interpretation slows down display.
- Variables declared in a JavaScript function become LOCAL to the function.
- Local variables can only be accessed from **within a function**. Because local variables are recognized only inside their functions, variables **with the same name** can be used **in different functions**.
- Local variables are created when the function starts and **deleted** when the function is **finished**.
- ```
// can't use the carName variable here
function MyCar() {
    let carName = "Volvo";
    // variable carName can be used here
}
// can't use the carName variable here
```

Functions - example

```
<!DOCTYPE html>
<html>
  <!--file: function2.html, content
from: www.w3schools.com-->
  <head>
    <title>Function 2</title>
  </head>
<body>
<h2>JavaScript Functions</h2>
<p>After a function has been stored in
a variable, the variable can be used as
a function:</p>
<p id="demo"></p>
<script>
const x = function (a, b) {return a +
a* b};
document.getElementById("demo").innerHT
ML = x(5, 6);
</script>

</body>
</html>
```



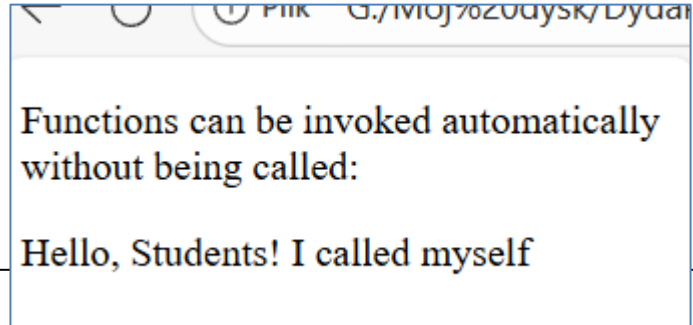
```
<!DOCTYPE html>
<html>
  <!--file: function4.html , content
from: www.w3schools.com-->
  <head>
    <title>Function 4</title>
  </head>
<body>
  <h2>JavaScript Functions</h2>
  <p>JavaScript has an built-in
function constructor.</p>
  <p id="demo"></p>
  <script>
    const myFunction = new Function("a",
    "b", "return a + a * b");
    document.getElementById("demo").inner
    HTML = myFunction(5, 6);
  </script>
</body>
</html>
```



"Self-calling" functions

- JavaScript functions can be called before declaring them
- Function expressions can also be "self-calling".
- Function expressions will be executed automatically if the expression follows `()`.
- You cannot independently call such a declared function.
- This function is actually an anonymous self-calling function (an unnamed function).
- An example: `function1.html`

```
<body>
<p>Functions can be invoked automatically without being called:</p>
<p id="demo"></p>
<script>
(function () {
    document.getElementById("demo").innerHTML = "Hello, Students! I called myself";
})();
</script>
</body>
</html>
```



Functions can be invoked automatically
without being called:

Hello, Students! I called myself

Functions are objects

- The `typeof` operator in JavaScript returns a "function" for a function.
- However, JavaScript functions can best be described as objects.
- JavaScript functions have both **properties** and **methods**
 - The `arguments.length` property returns the number of arguments received when the function is called:

```
function myFunction(a, b) {  
    return arguments.length;  
}
```
- The `toString()` method returns a function as a string.
- A **function** defined as an **object property** is called an **object method**.
- A **function** designed to **create** new objects is called an **object constructor**.

Functions – example 2

```
<!DOCTYPE html>
<html>
  <!--file: function5.html, content
from: www.w3schools.com-->
  <head>
    <title>Function 5</title>
  </head>
<body>
  <h2>JavaScript Functions</h2>
  <p>The arguments.length property
returns the number of arguments
received by the function:</p>
<p id="demo"></p>
<script>
function myFunction(a, b) {
  return arguments.length;
}
document.getElementById("demo").innerHT
ML = myFunction(5, 6);
</script>
</body>
</html>
```

JavaScript Functions

The arguments.length property returns the number of arguments received by the function:

2

```
<!DOCTYPE html>
<html>
  <!--file: function3.html, content
from: www.w3schools.com-->
  <head>
    <title>Function 3</title>
  </head>
<body>
  <h2>JavaScript Functions</h2>
  <p>The toString() method returns the
function as a string:</p>
<p id="demo"></p>
<script>
function myFunction(a, b) {
  return a * a + b;
}
document.getElementById("demo").innerHT
ML = myFunction.toString();
</script>
</body>
</html>
```

JavaScript Functions

The toString() method returns the function as a string:

```
function myFunction(a, b) { return a +
a* b; }
```

Functions as object-oriented methods

- In JavaScript, you can define functions as object-oriented methods.
- The following example creates an object (`myObject`) with two properties (`firstName` and `lastName`) and a method (`fullName`):

```
const myObject = {  
  firstName: "Anna",  
  lastName: "Jasna",  
  fullName: function () {  
    return this.firstName + " " + this.lastName;  
  }  
}  
myObject.fullName(); // Will return "Anna Jasna"
```

- The `fullName` method is a function. The function belongs to the object.
 - `myObject` owns the function.
- A thing called `this` is an object that "owns" the JavaScript code. In this case, the value of `this` is `myObject`.

Calling a function by using the function constructor

- If the function call is preceded by the `new` keyword, it is a constructor call.
- It looks like you're creating a new function, but because JavaScript functions are objects, you're actually creating a **new object**
- A **constructor call** creates a **new object**. The new object **inherits properties and methods** from its constructor.
- The keyword `this` in the constructor has no value. The `this` value will be the **new object** created when the function is called.

```
// This is a function
constructor:
function myFunction(arg1,
arg2) {
    this.firstName = arg1;
    this.lastName  = arg2;
}

// This creates a new object
const myObj
= new myFunction("Anna", "Jasna");

// This will return "Anna"
myObj.firstName;
```

Constructor for functions - example

- An example: `constr.html`

```
<body>
<h2>JavaScript Object Constructors</h2>
<p>You cannot add a new property to a constructor function.</p>
<p id="demo"></p>
<script>
// Constructor function for Person objects
function Person(first, last, age, eye) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
  this.eyeColor = eye;
}
// You can NOT add a new property to a constructor function
Person.nationality = "Polish";
// Create 2 Person objects
const myFather = new Person("Jan", "Nowak", 50, "blue");
const myMother = new Person("Anna", "Jasna", 48, "green");
// Display nationality
document.getElementById("demo").innerHTML = "The nationality of my father is " + myFather.nationality;
// You can add a new property to a constructor function:
Person.prototype.nationality = "Polish";
//
Person.prototype.name1 = function() {
  return this.firstName + " " + this.lastName
};
document.getElementById("demo").innerHTML += "<br> The nationality of my mother " + myMother.name1() + " is "
+ myMother.nationality;
</script>
</body></html>
```

JavaScript Object Constructors

You cannot add a new property to a constructor function.

The nationality of my father is undefined
The nationality of my mother Anna Jasna is Polish

prototype property

- The prototype property allows you to add new properties (or methods) to all existing objects of the type.
- The JavaScript prototype property allows you to add **new properties** (or methods) to
- Modify only custom prototypes. Never modify prototypes of standard JavaScript objects.

```
<!DOCTYPE html>
<html>
  <!--File: prototype.html, content from: www.w3schools.com -->
  <head>
    <title>Prototype</title>
  </head>
  <body>
    <h2>Prototype</h2> <!--The prototype property allows you to add
    new methods to objects constructors:
    -->
    <p id="demo"></p>
    <script>
      function Man(fname, lname, age, eye) {
        this.firstName = fname;
        this.lastName = lname;
        this.age = age;
        this.eyeColor = eye;
      }
      Man.prototype.nationality = "Polish";
      Man.prototype.name = function() {
        return this.firstName + " " + this.lastName
      };
      const myFather = new Man("Jan", "Nowak", 48, "blue");
      document.getElementById("demo").innerHTML =
        "My father is " + myFather.name() + "<br>" +
        "The nationality of my father is " + myFather.nationality;
    </script>
  </body></html>
```

Prototype

My father is Jan Nowak
The nationality of my father is Polish

All functions are methods

- In JavaScript, **all functions** are object-oriented **methods**.
- If the function is not a JavaScript object method, it is a function of the **global object**.
 - In the example, `this` refers to the `person` object.
 - `this.firstName` is the `firstName` property for `this`.
 - But also `this.firstName` stands for the `firstName` property for `person`.

```
const person = {
  firstName: "Anna",
  lastName: "Jasna",
  fullName: function () {
    return this.firstName +
      " " + this.lastName;
  }
}

// This will return "Anna
// Jasna":
person.fullName();
```

Method `call()`

- The `call()` method is a predefined JavaScript method.
- It can be used to call a method with an owner object as an argument (parameter).
- With `call()`, an object can use a method that belongs to another object.

```
const person = {
  fullName: function() {
    return this.firstName + "
" + this.lastName;
  }
}
const person1= {
  firstName:"Jan",
  lastName: "Nowak"
}
const person2 = {
  firstName:"Anna",
  lastName: "Jasna"
}
// This will return "Jan
Nowak":
person.fullName.call(person1);
```

Methods `apply()` and `call()`

- You can use the `apply()` method to write a method that can be used on different objects.
- The `apply()` method is similar to the `call()` method.
- The difference between `call()` and `apply()` is:
 - The `call()` method gets the arguments separately.
 - The `apply()` method takes arguments as an array.
- The `apply()` method is very useful if you want to use an array instead of a list of arguments.
- The largest number (in the list of numbers) can be found using the `Math.max()` method
- Because JavaScript arrays do not have a `max()` method, you can use this `Math.max()` method instead.
 - `Math.max.apply(null, [1,2,3]); // Will also return 3`
- The first argument (`null`) is irrelevant. Not used in this example.
- These examples will give the same result:
 - `Math.max.apply(Math, [1,2,3]); // Will also return 3`

Methods apply() and call() - examples

- Examples: call1.html, apply1.html

```
<body>
<h2>JavaScript Functions</h2>
<p>This example calls the fullName method of
osoba, using it on osoba1 and osoba2:
</p>

<p id="demo"></p>

<script>
const osoba = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName +
    "," + city + "," + country;
  }
}
const osoba1 = {
  firstName: "Jan",
  lastName: "Nowak"
}
const osoba2 = {
  firstName: "Anna",
  lastName: "Jasna"
}

document.getElementById("demo").innerHTML =
osoba.fullName.call(osoba1, "Wrocław", "Polska") +
"<br>" + osoba.fullName.call(osoba2, "Warszawa",
"Polska"); ;
</script>
</body></html>
```

JavaScript Functions

This example calls the fullName method of osoba, using it on osoba1 and osoba2:

Jan Nowak,Wrocław,Polska
Anna Jasna,Warszawa,Polska

```
<body>
<h2>JavaScript Functions</h2>
<p>W tym przykładzie fullName metoda osoba jest
<b>zastosowana</b> do osoba1:</p>

<p id="demo"></p>
<script>
const osoba = {
  fullName: function(city, country) {
    return this.firstName + " " + this.lastName +
    "," + city + "," + country;
  }
}
const osoba1 = {
  firstName: "Anna",
  lastName: "Nowak"
}

document.getElementById("demo").innerHTML =
osoba.fullName.apply(osoba1, ["Wrocław",
"Polska"]) + "<br>" + osoba.fullName.call(osoba1,
"Wrocław", "Polska");
</script>
</body></html>
```

JavaScript Functions

W tym przykładzie fullName metoda osoba jest **zastosowana** do osoba1:

Anna Nowak,Wrocław,Polska
Anna Nowak,Wrocław,Polska

Borrow functions

- With the `bind()` method, an object can borrow a method from another object.
- Example: `bind.html`

JavaScript Function `bind()`

This example will try to display a osoba name after 2 seconds (wrong) and 4 second (correct).

Anna Nowak

JavaScript Function `bind()`

This example will try to display a osoba name after 2 seconds (wrong) and 4 second (correct).

undefined undefined

JavaScript Function `bind()`

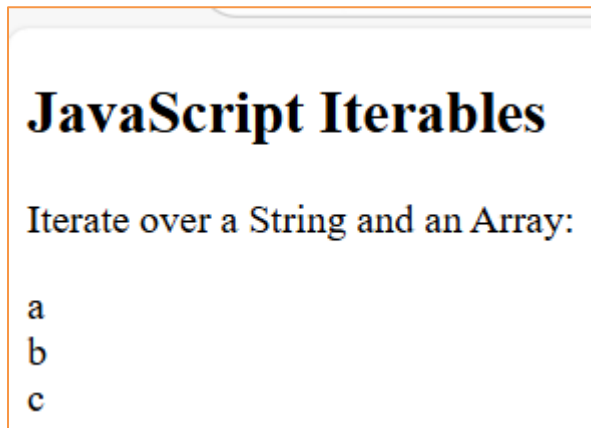
This example will try to display a osoba name after 2 seconds (wrong) and 4 second (correct).

Anna Nowak

```
<body>
<h1>JavaScript Function bind()</h1>
<p>This example will try to display a osoba name
after 2 seconds (wrong) and 4 second (correct).</p>
<p id="demo"></p>
<script>
const person = {
  firstName: "Anna",
  lastName: "Nowak",
  display: function() {
    let x = document.getElementById("demo");
    x.innerHTML = this.firstName + " " +
this.lastName;
  }
}
// 1) displays correctly
person.display();
// 2) displays undefined because if a function is
used as a callback, this is lost.
setTimeout(person.display, 2000);
// 3) the bind() method is used to bind
person.display to person
setTimeout(person.display.bind(osoba), 4000);
</script>
</body>
</html>
```

Iterating

- Iterable objects are objects that you can iterate over by using the `for (... of ...)` - iteration over sequence of elements.
- Example: `iter2.html`



```
<body>
<h2>JavaScript Iterables</h2>
<p>Iterate over a String and an Array:</p>

<p id="demo"></p>
<script>
  // Create a new String
  const name = "W3Schools";
  iter(name);

  // Create a new Array
  const letters = ["a", "b", "c"];
  iter(letters);

  function iter(text2) {
    text = "";
    // List all Elements
    for (const x of text2) {
      text += x + "<br>";
      wait(2000, text);
    }
    function wait(time1, text3){
      setTimeout(function(){
        alert("2 seconds have passed.");
        document.getElementById("demo").innerHTML = text3;},
        time1);
    }
  }
</script>
</body></html>
```

Set/Collection

- A JavaScript `Set` is a set of unique values.
- Each value can occur only once in a set.
- A set can store any value of any data type.

Method	Description
<code>new Set()</code>	Creates a new Set
<code>add()</code>	Adds a new element to the Set
<code>delete()</code>	Removes an element from a Set
<code>has()</code>	Returns true if a value exists in the Set
<code>forEach()</code>	Invokes a callback for each element in the Set
<code>values()</code>	Returns an iterator with all the values in a Set
Property	Description
<code>size</code>	Returns the number of elements in a Set

Sets - example

- Creation a JavaScript Set:
 - Passing an array to `new Set()`
 - Sequence of `add()` methods after creating an object
- If you add equal elements, only the first one will be saved
- The set has no keys. So method `entries()` returns `[value, value]` pairs instead of `[key, value]` pairs.
- Example: `set.html`

JavaScript Sets

a
c
d
e

4 Elements

A letter a is in the set.

a,a
c,c
d,d
e,e

```
<body>
<h2>JavaScript Sets</h2>
<p id="demo"></p>
<p id="demo1"></p>
<p id="demo2"></p>
<script>
// Create a Set
const letters = new Set(["a","b","c"]); // Creates a new Set
letters.add("a"); //Adds a new element to the Set
letters.add("d");
letters.add("c");
letters.add("e");
letters.delete("b"); // Removes an element from a Set
// List all Elements
let text = "";
letters.forEach (function(value) { //Invokes a callback for each
element
    text += value + "<br>";
})
text = text + "<br>"+letters.size + " Elements"; //Returns true if a
value exists
document.getElementById("demo").innerHTML = text;
letter="a";
text ="A letter "+letter +" is "
if (letters.has(letter)) {text+= " ";} //Returns true if a value
exists
else {text+= "not ";};
document.getElementById("demo1").innerHTML = text + "in the set.";

// List all entries
const iterator = letters.entries(); //returns [value,value] pairs
instead of [key,value] pairs.
text = "";
for (const entry of iterator) {
    text += entry + "<br>";
}
document.getElementById("demo2").innerHTML = text;
</script>
</body></html>
```

"use strict" mode 1/2

- The developers of JavaScript have added a **new feature** in ES5 known as **strict mode**, which is designed to prevent specific language behaviors to **reduce random behavior** and **increase the detectability** of poorly written code. This set of limitations has made the code much **more secure** and generally maintains a **high standard of encoding**.
- This mode is much more obsessed with correct syntax and other logical paradigms.:
 - Auto-global variable declaration: this is one of the biggest problems in JavaScript, without using strict mode; if you mistakenly use a variable without defining it, not only will JavaScript not generate an error, but it will declare the variable in global scope. This often leads to randomness and adverse reactions or output.
 - Removal of any JavaScript element: This is a big change from normal mode because no variable or function can be deleted in strict mode. This makes the code much more optimized because the ranges are static and do not change during the lifetime.
 - Using reserved keywords as variable names: Unlike most other languages, JavaScript allowed the use of reserved keywords as variable names that are not allowed in strict mode.
 - Duplicate parameter names: Unlike most other languages, JavaScript allowed the use of duplicate parameter names, which is disabled in strict mode.

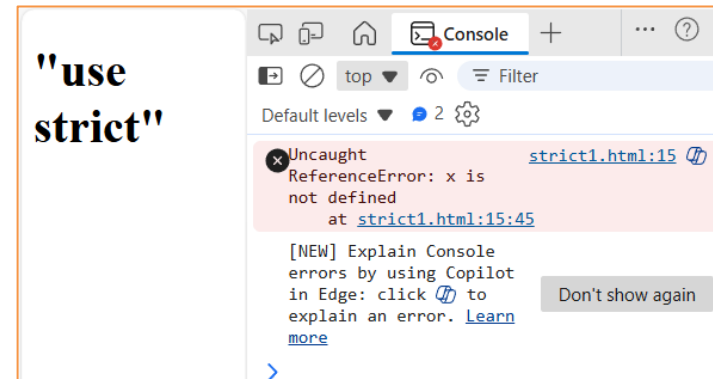
"use strict" mode 2/2

- "use strict" - specifies that JavaScript code should be executed in "strict mode".
 - This is **not a statement**, but a literal expression, **ignored** by **earlier** versions of JavaScript.
- The purpose of "use strict" is to indicate that the code should be executed in "strict mode".
 - For example, you cannot use undeclared variables in strict mode.
- Strict mode is declared by adding "use strict" to the **beginning of a script or function**.
- Declared at the beginning of the script, it has a **global scope** (all code in the script will be executed in strict mode).
- Remark! The "use strict" directive is **recognized only** at the **beginning** of a script or function.

Why strict mode?

- Strict mode makes it easy to write "secure" JavaScript.
- Strict mode turns a previously accepted "bad syntax" into actual errors.
- For example, in normal JavaScript, mistyping a variable name creates a new global variable. In strict mode, this will report an error, preventing the accidental creation of a global variable.
- In normal JavaScript, the programmer will not receive any feedback about errors in assigning values to properties that cannot be saved. In strict mode, any assignment to an unsavable property, a get-only property, a non-existent property, a non-existent variable, or an object that does not exist will result in an error.
- Example: `strict1.html`

```
<body>
<h1>"use strict"</h1>
<p id="demo"></p>
<script>
"use strict"; //you can try to delete
// let x = 4; // you can try to remove let
document.getElementById("demo").innerHTML = x;
</script>
</body>
</html>
```



Not allowed in strict mode 1/2

- Using a variable without declaring it.
- Working with an object without declaring it.
- Deleting a variable (or object) is not allowed.

```
"use strict";  
let x = 3.14;  
delete x; // This will cause an error
```

- Removal of function is not allowed.

```
"use strict";  
function x(p1, p2) {};  
delete x; // This will cause an error
```

- Duplication of parameter name is not allowed:

```
"use strict";  
function x(p1, p1) {}; // This will cause an error
```

- Octal literals are not allowed:

```
"use strict";  
let x = 010; // This will cause an error
```

- Signs of octal escape are not allowed.
- Writing to read-only properties is not allowed.

Not allowed in strict mode 2/2

- The keyword `this` in functions behaves differently in strict mode.
- The keyword `this` refers to the object that called the function.
- If the object is not specified, functions in strict mode return, `undefined`, but functions in normal mode return a global object (`window`).
- Keywords (including those reserved for future versions of JavaScript) CANNOT be used in strict mode as variable names:
 - `implements`
 - `interface`
 - `let`
 - `package`
 - `private`
 - `protected`
 - `public`
 - `static`
 - `yield`

Keyword `this` 1/3

- The `this` keyword in JavaScript has often been a source of confusion for beginners in the language. Part of this confusion is due to the fact that JavaScript treats it **differently than other languages**, such as Java or `self` in Python.
- **Functions** in JavaScript are essentially **objects**. Like objects, they can be assigned to variables, passed to other functions, and returned from functions. And like objects, they have their **own properties**. One of these **properties** is `this`.
- The value that `this` stores is the **current execution context** of the JavaScript program. So, when it is used inside a function, this value will change depending on how the function is defined, how it is called, and the **default execution context**.
- **Note:** `this` always contains a **reference to a single object** that defines the **current line** of the code **execution context**.

Keyword `this` 2/3

- Global context:
 - the line of code written **outside the function** is to belong to the **global context**, and the value of `this` in this global context is **the same as the global object**.
 - For example, if you open the browser console and type the following lines into it and press `<enter>`:
`console.log(this)`
You will see that the `window` object is shown on the console. This is because the **global object** in the browser runtime, such as the Chrome runtime, is the `window` object.
 - However, **inside the function**, the global context may **no longer be present**, and the function may have its **own defined context** and therefore a different value.

Keyword `this` 3/3

- In JavaScript, `this` keyword refers to the object . Which object depends on how this is called (used or called). The `this` keyword refers to different objects depending on how it is used
 - In the object method, this refers to the object .
- In the global function, this refers to a global object. When you call a function as a global function, its value is a global object. Using a `window` object as a variable can easily cause the program to crash.
- In the function in **strict mode**, in the above cases, `this` is `undefined`.
- In an **event**, `this` refers to the element that **received the event**.
- Methods such as `call()`, `apply()`, and `bind()` can refer `this` to any object.
- `this` is not a variable. It's a keyword. You **can't change** the value of `this`.

Keyword *this* for an event

- In HTML **event handling**, *this* refers to the HTML element **that received the event**:
 - Example: `thisOnClick.html`

```
<!DOCTYPE html>
<html>
  <!--file: thisOnClick.html, content from: www.w3schools.com-->
  <head>
    <title>This - onClick</title>
  </head>
  <body>

    <h1>Keyword this for an event onClick.</h1>

    <button onclick="this.style.display='none'">Click me to remove 1!</button>
    <button onclick="this.style.display='none'">Click me to remove 2!</button>
    <button onclick="this.style.display='none'">Click me to remove 3!</button>
  </body>
</html>
```

Keyword *this* for an event onClick.

Click me to remove 1!

Click me to remove 2!

Click me to remove 3!

Keyword *this* for an event onClick.

Click me to remove 2!

Click me to remove 3!

Keyword `this` in global context

- Example: `thisWindow.html`

```
<!DOCTYPE html>
<html>
  <!--file: thisWindow.html, content from: www.w3schools.com-->
  <head>
    <title>This - window</title>
  </head>
  <body>
    <h1>Keyword this in JavaScript</h1>
    <p>In this example this refers to the object Window:</p>
    <p id="demo1"></p>
    <p id="demo2"></p>
    <script>
      let x = this;
      document.getElementById("demo1").innerHTML = x;
      document.getElementById("demo2").innerHTML = myFunction();
      function myFunction() {
        return this;
      }
    </script>
  </body>
</html>
```

Keyword *this* in JavaScript

In this example **this** refers to the object **Window**:

[object Window]

[object Window]

Keyword *this* in an object

- Example: `thisPerson.html`

```
<body>
<h1>Keyword this in object person</h1>
<p>Example of this, which refers to the person object.</p>
<p>Because fullName is a method of the person object.</p>
<h2>fullName</h2>
<p id="demo1"></p>
<h2>this</h2>
<p id="demo2"></p>
<script>
const person = {
  firstName: "Anna",
  lastName: "Nowak",
  id: 1122,
  fullName : function() {
    return this.firstName + " " + this.lastName;
  },
  myFunction : function() {
    return this;
  }
};
document.getElementById("demo1").innerHTML = person.fullName();
document.getElementById("demo2").innerHTML = person.myFunction();
</script>
</body>
</html>
```

Keyword *this* in object person

Example of **this**, which refers to the **person** object.

Because **fullName** is a method of the person object.

fullName

Anna Nowak

this

[object Object]

Arrow function (lambda expressions)

- Arrow functions allow us to write a shorter syntax for functions:

```
let myFunction = (a, b) => a * b;
```

- If the function has only **one statement** and the statement **returns a value**, you can remove the parentheses and the `return` keyword:

```
hello = () => "Hello World!";
```

```
hello2 = val => "Hello " + val;
```

- `this` support is also different in the arrow functions compared to the usual functions.
- The arrow functions do not have a normal binding `this`.
- For arrow functions, the `this` keyword always represents the **object that defined** the arrow function.

Classes (from ES6)

- In JS, a **class** is an object generator (prototyper).
- Use the `class` keyword to create a class
- The construction of the object is carried out by the operator **new**. The `constructor()` method is then started, which can be **only one**.
 - Always add the `constructor()` method
- **Class methods** are created using **the same syntax** as **object-oriented methods**
- Once created, everything can still be exchanged in the object!
 - It cannot be said that the `obj` object is of the Class `class` type, even as it was created from its constructor.
 - But information about the prototype (class) is available

```
class Person{
  constructor(name,salary){
    this.name=name;
    this.salary=salary;
  }
  getFormatted(separator){
    let result=this.name+separator+this.salary;
    //let result="" +this.name+separator+this.salary;
    return result;
  }
}
let person=new Person("Einstein",2000);
console.log(person);
console.log(person.getFormatted(" -> "));
let number=new Person(1,2);
console.log(number.getFormatted(3)); // plus for numbers
```

► Person {name: 'Einstein', salary: 2000}	class.html:24
Einstein -> 2000	class.html:25
6	class.html:27

uncommented

► Person {name: 'Einstein', salary: 2000}	class.html:24
Einstein -> 2000	class.html:25
132	class.html:27

Function call in an object created from a class

- In general, a **function** with a **given name** in an object is **first searched** for and will be executed.
- If there is none, the function is searched in the prototype for the class from which the object was created.
- Then possibly in the parent class. Etc.
- If it is not found anywhere – an exception for execution
- Below: Objects have a lot of information in the `prototype` property.

```
▼ Person {name: 'Einstein', salary: 2000} ⓘ class.html:24
  name: "Einstein"
  salary: 2000
  ▼ [[Prototype]]: Object
    ▼ constructor: class Person
      arguments: (...)
      caller: (...)
      length: 2
      name: "Person"
      ▼ prototype:
        ► constructor: class Person
        ► getFormatted: f getFormatted(separator)
        ► [[Prototype]]: Object
        ► [[FunctionLocation]]: class.html:13
        ► [[Prototype]]: f ()
        ► [[Scopes]]: Scopes[2]
        ► getFormatted: f getFormatted(separator)
        ► [[Prototype]]: Object
    Einstein -> 2000 class.html:25
    132 class.html:27
```

Class inheritance

- To create class inheritance, use the `extends` keyword.
- A class created with class inheritance inherits all methods from another class.
- The `super()` method refers to the parent class.
 - By calling the `super()` method in the constructor method, we call the parent constructor method.
 - By calling the `super()` method we access the properties and methods of the parent.
- Example: `inherit.html`

```
<body>
<h1>JavaScript Class Inheritance</h1>
<p>Use the "extends" keyword to inherit all methods from another class.</p>
<p>Use the "super" method to call the parent's constructor function.</p>
<p id="demo"></p>
<script>
class Person {
  constructor(name) {
    this.name = name;
  }
  show() {
    return 'My name is ' + this.name;
  }
}
class Worker extends Person {
  constructor(name, monthSalary) {
    super(name);
    this.salary = monthSalary;
  }
  show() {
    return super.show() + ', my salary is ' + this.salary;
  }
}
let worker1 = new Worker("Newman", 2000);
document.getElementById("demo").innerHTML = worker1.show();
</script>
</body>
```

JavaScript Class Inheritance

Use the "extends" keyword to inherit all methods from another class.

Use the "super" method to call the parent's constructor function.

My name is Newman, my salary is 2000

JavaScript modules

- JavaScript modules allow you to break down code into separate files.
- JavaScript modules are based on `import` and `export` statements.
- **Modules work only with the HTTP(s) protocol. A website opened using the `file://` protocol cannot use `import/export`.**
- You can export a function or variable to any file. There are two types of export: named and default.
- You can create named exports in two ways.
- In-line **one at a time**

```
export const firstName = "Anna";  
export const lastName = "Jasna";  
export const age = 30;
```
- or all at once **at the bottom**

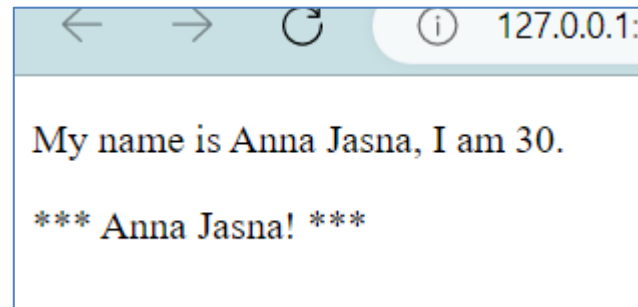
```
const firstName = "Anna";  
const age = 30;  
export {firstName, age};
```

Export/import example

- Example: import2.html, person.js

```
<!DOCTYPE html>
<html lang="en">
  <!--File:export2.html-->
<head>
  <title>Import</title>
</head>
<body>
<p id="demo">demo1</p>
<p id="demo2">demo2</p>
  <script type="module">
    import { firstName, lastName, age } from "./person.js";
    let text = "My name is " + firstName + " " + lastName + ", I am " + age + ".";
    document.getElementById("demo").innerHTML = text;
    import { withStars } from "./person.js";
    document.getElementById("demo2").innerHTML = withStars(firstName + " " + lastName);
  </script>
</body>
</html>
```

```
//File: person.js
export const firstName = "Anna";
export const lastName = "Jasna";
export const age = 30;
export function withStars(line) {
  return `*** ${line}! ***`;
}
```



Getters and setters in the class

- Classes also allow you to use getters and setters.
- To add getters and setters in a class, use the `get` and `set` keywords.
- Note: Even if getter is a method, do not use parentheses when you want to get the value of the property.
 - Use like a property/field
- The name of the getter/setter method cannot be the same as the name of the property, in this case `Brand`.
- Many developers use the `'_'` underscore sign before the property name to separate the getter/setter method from the actual property/

Getter and setter example

- Example: getter4.html

```
<title>Gettter & Setter</title>
</head>
<body>
<h2>JavaScript Class Gettter/Setter</h2>
<p>A demonstration of how to add getters and setters in a class,
  and how to use the getter to get the property value.</p>
<p id="demo"></p>
<script>
class Car {
  constructor(brand) {
    this._Brand = brand; //underscore before property name
  }
  get Brand() { //getter
    return this._Brand;
  }
  set Brand(x) { //setter
    this._Brand = x;
  }
}
let myCar = new Car("Ford"); //set property to "Ford"
myCar.Brand = "Toyota"; //use of setter - change of property to "Toyota"
document.getElementById("demo").innerHTML = myCar.Brand; // use of getter
</script>
</body>
</html>
```

JavaScript Class Gettter/Setter

A demonstration of how to add getters and setters in a class, and how to use the getter to get the property value.

Toyota

Classes vs hoisting

- Unlike functions and other JavaScript declarations, **class declarations are not hoisted**.
- This means that you must **declare** a class **before** you can **use** it.
- Note: For other declarations, such **as functions**, you will **not** get an **error** when you try to use it before declaring, because the **default behavior** of a JavaScript **declaration** is **hoisting**.

Static methods

- Static methods are defined in the class itself.
 - Add keyword `static` before method's name.
- You cannot call a static method on an object, only on an object class.
- Example: `stat1.html`

```
<body>
<h2>JavaScript Class Static Methods</h2>
<p>A static method is created with the "static" keyword,
  and you can only call the method on the class itself.</p>
<p id="demo"></p>
<script>
class Car {
  constructor(name) {
    this.name = name;
  }
  static hello() { // Static method
    return "Hello!! "+this.name;
  }
}
let myCar = new Car("Toyota");
//You can call 'hello()' on the Car Class:
document.getElementById("demo").innerHTML = Car.hello();
// But NOT on a Car Object:
// document.getElementById("demo").innerHTML = myCar.hello();
// this will raise an error:
// Uncaught TypeError: myCar.hello is not a function
</script>
</body>
```

JavaScript Class Static Methods

A static method is created with the "static" keyword, and you can only call the method on the class itself.

Hello!! Car

Callback function

- **Callback** is a function passed as an argument to another function
- This technique allows a function to call another function
- In the example above, `myDisplayer` is the name of the function, while in the `myCalculator()` function, it is passed as an argument.
 - A function passed as an argument is written without parentheses.
- Example: `callback2.html`

```
<body>
<h2>JavaScript Callbacks</h2>
<p>Do a calculation and then display the result.</p>
<p id="demo"></p>
<script>
function myDisplayer(something) {
    document.getElementById("demo").innerHTML = something;
}

function myCalculator(num1, num2, myCallback) {
    let sum = num1 + num2;
    myCallback(sum);
}
myCalculator(5, 5, myDisplayer); // name of myDisplayer as argument to function myCalculator
</script>
</body>
```

JavaScript Callbacks

Do a calculation and then display the result.

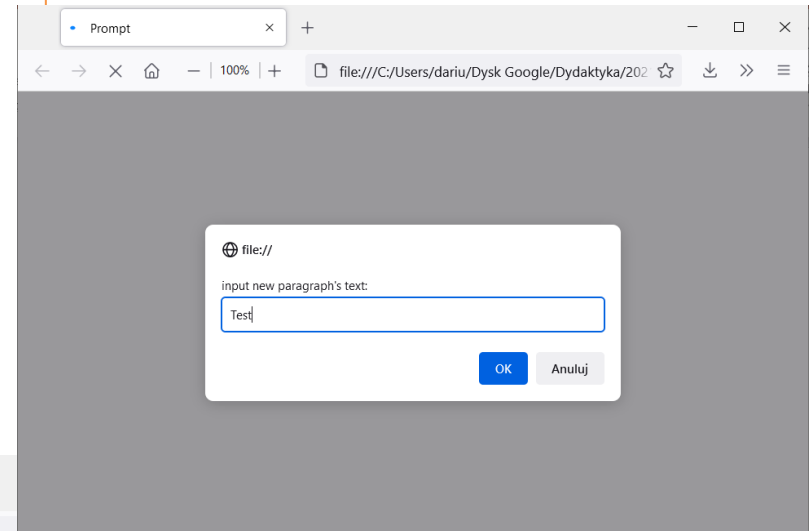
10

JavaScript and the HTML page

- Certain objects are always present for JavaScript code in a web page and have a lot of properties and functions:
 - `window` – object with window data (tab) with HTML document
 - `document` (copy of `window.document`) – to manage HTML document elements
- Simple communication with the user (it was used before):
 - `window.prompt(QuestionString, defaultValue)`
 - Opens a window asking for text, returns the typed string.
 - `pElem=document.getElementById(elementId)`
 - Reference to an object of an element with a given identifier or `undefined`
 - field `pElem.innerHTML` – when you assign a new value to test, rendering of it as an HTML fragment. Can be read.
 - field `pElem.innerText` – after assigning exactly the same internal text of the element (with possible change to special characters). Can be read.
- Example:
 - `prompt.html` - The execution ends with an exception because the elements have not yet been rendered (the script is too early). The page contains the initial values of the elements `#p1` i `#myH1`
 - `prompt2.html` – The script works correctly by inserting new values into elements `#p1` and `#myH1`

Example – prompt.html

```
<title>Prompt</title>
</head>
<body>
  <script>
    var text1=window.prompt("input new paragraph's text:","example");
    var pElem=document.getElementById("p1");
    pElem.innerHTML=text1; // tag rendering
    var text1=window.prompt("input new heading text:","Main");
    var pElem=document.getElementById("myH1");
    pElem.innerText=text1; // no rendering
    //console.log();
  </script>
  <h1>Prompt</h1>
  <p id="p1">
    Here will be an another text.
  </p>
  <h1 id="myH1"> Heading </h1>
  <p>
    Lorem ipsum (...).
  </p>
</body>
```

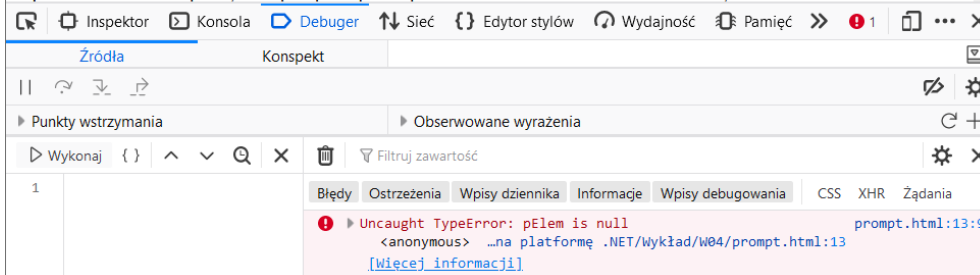


Prompt

Here will be an another text.

Heading

Lorem ipsum dolor sit amet consectetur adipisicing elit. Soluta eum nisi ex molestias. Perferendis labore repellendus sed numquam, cumque quos qui aliquam veritatis reiciendis amet laborum, at consectetur tenetur non.

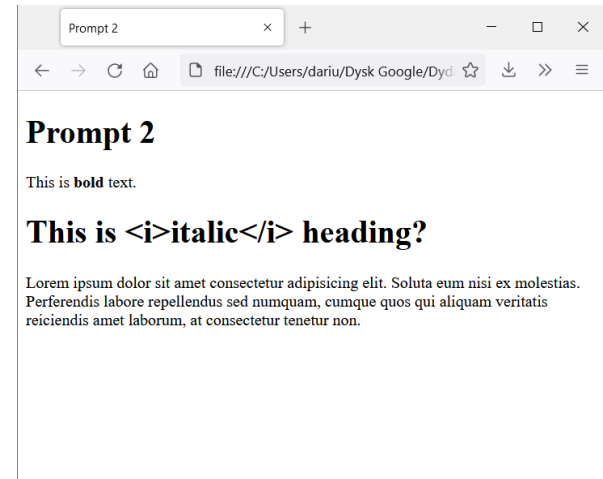
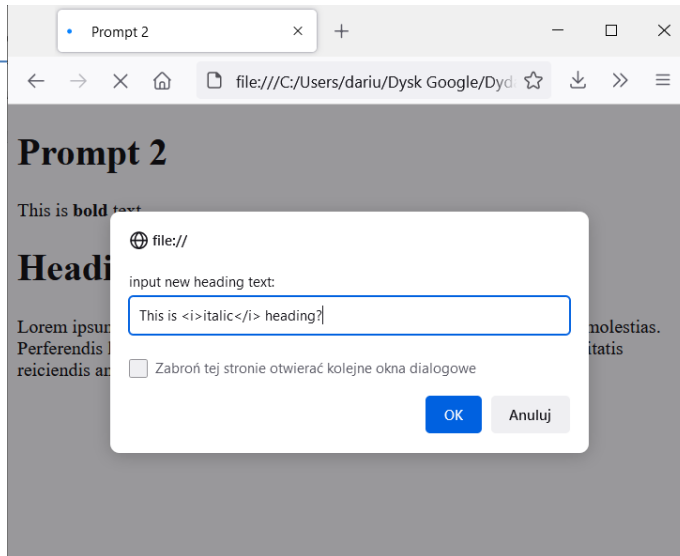
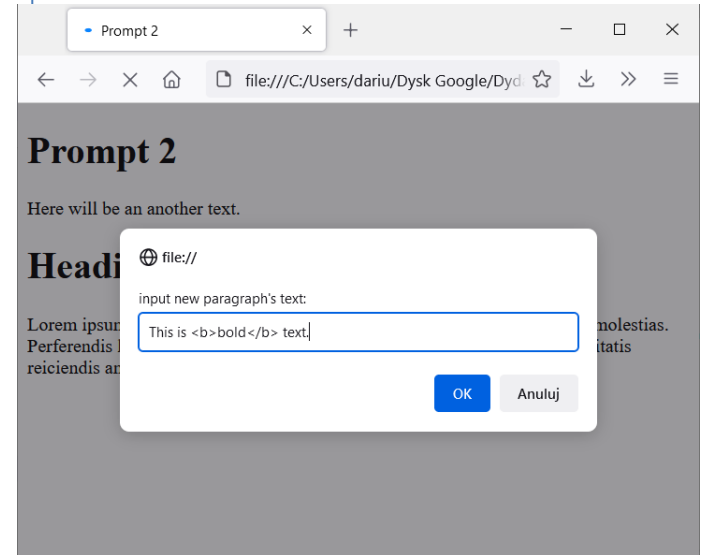


Before the script is completed, the rest of the page **is not** rendered, hence the #p1 element is not there yet!

You can move the script to the end of the document, but this is not always the best way out

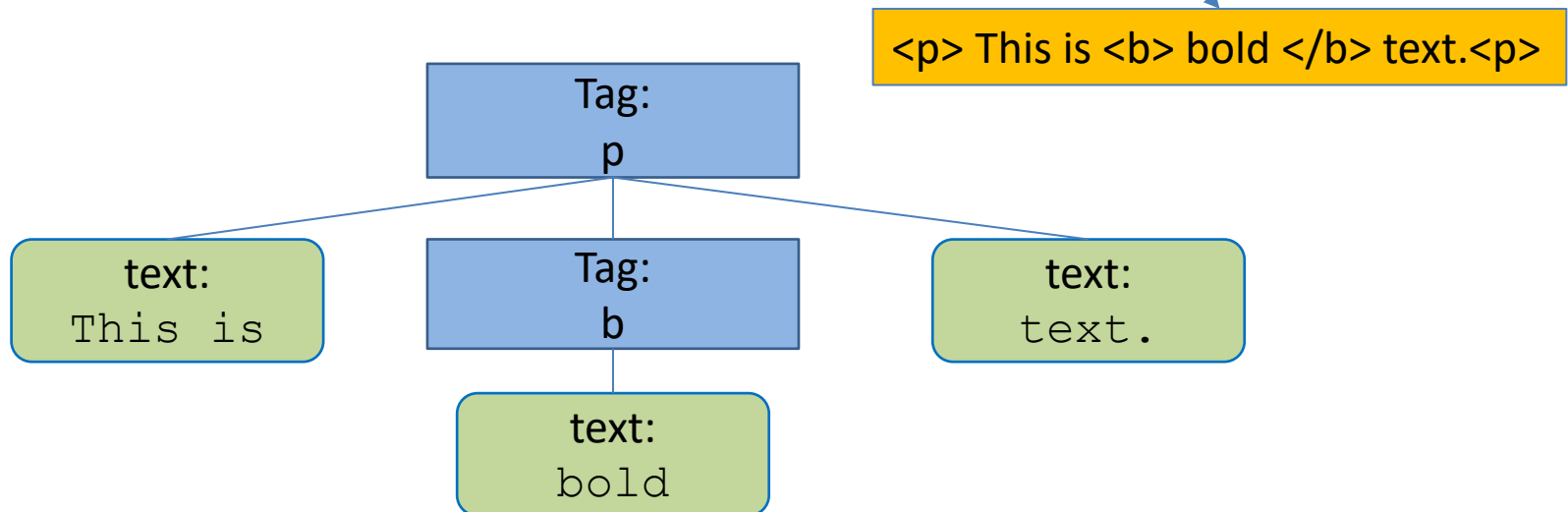
Example – prompt2.html

```
<title>Prompt 2</title>
</head>
<body>
  <h1>Prompt 2</h1>
  <p id="p1">
    Here will be an another text.
  </p>
  <h1 id="myH1"> Heading </h1>
  <p>
    Lorem ipsum dolor (...)
  </p>
  <script>
    var text1=window.prompt("input new paragraph's text:", "example");
    var pElem=document.getElementById("p1");
    pElem.innerHTML=text1; // tag rendering
    var text1=window.prompt("input new heading text:", "Main");
    var pElem=document.getElementById("myH1");
    pElem.innerText=text1; // no rendering
  </script>
</body>
```

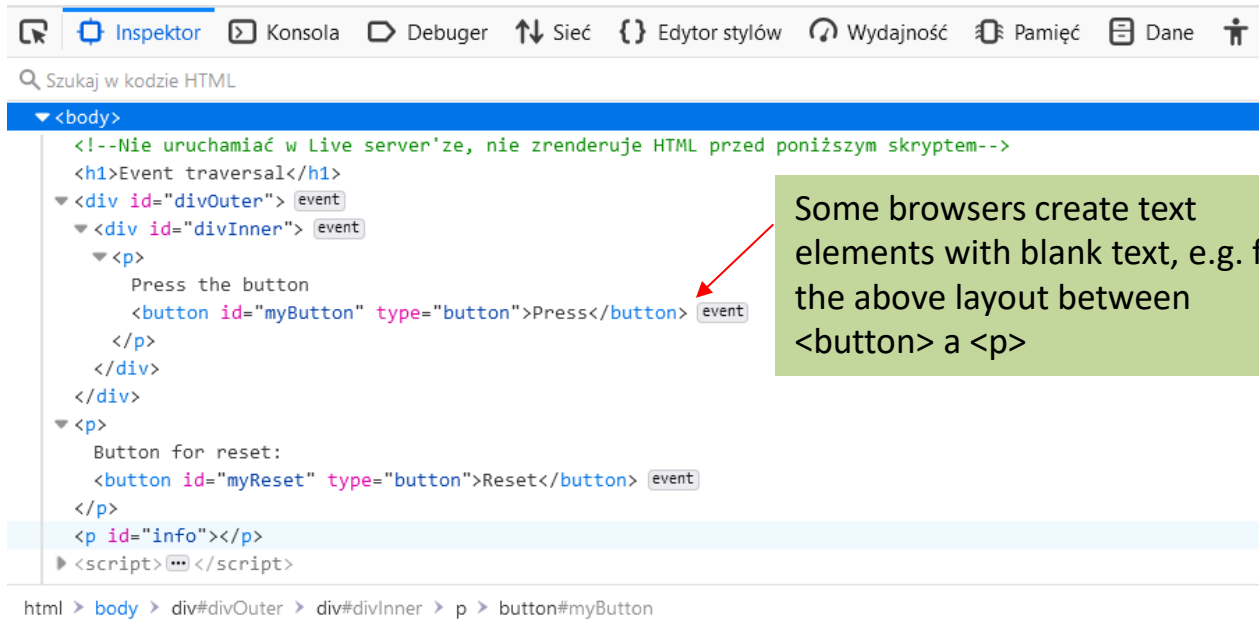


DOM, element attributes

- The Document Object Model (DOM) allows you to access all elements on the page.
- HTML elements (and not only, e.g. just text) make a tree.
- By using JavaScript, you can dynamically **create**, **modify** and **delete** elements from the page.
- Create a new tag
 - `document.createElement(elementName)`
- Create a new text element
 - `document.createTextNode(text)`
- There are methods to manipulate elements within the DOM:
 - Remove from the DOM model
 - Add to a parent as a child
 - At the beginning of the list of children, at the end, after the selected child, etc.
 - Move branches to a different location in the tree
 - Access to the parent
 - Access to the childrens



DOM model in the development tool and VS Code

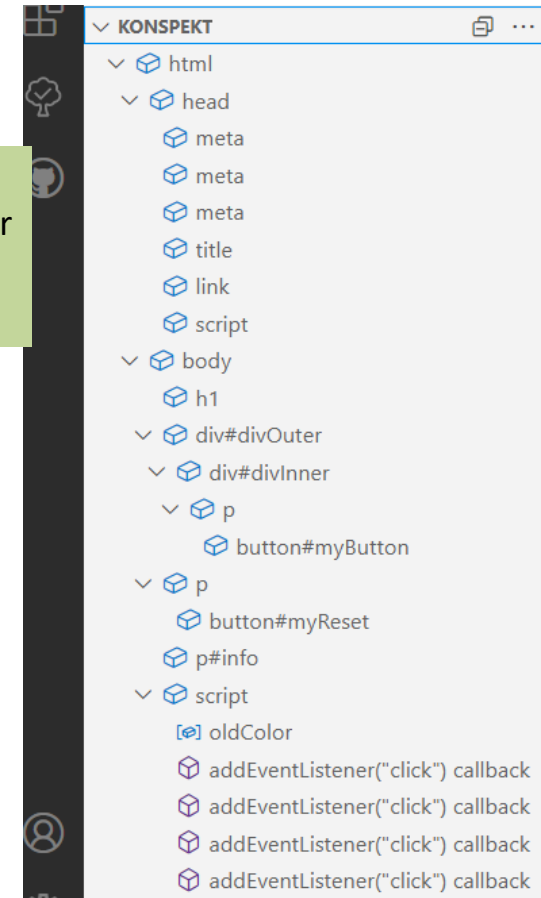
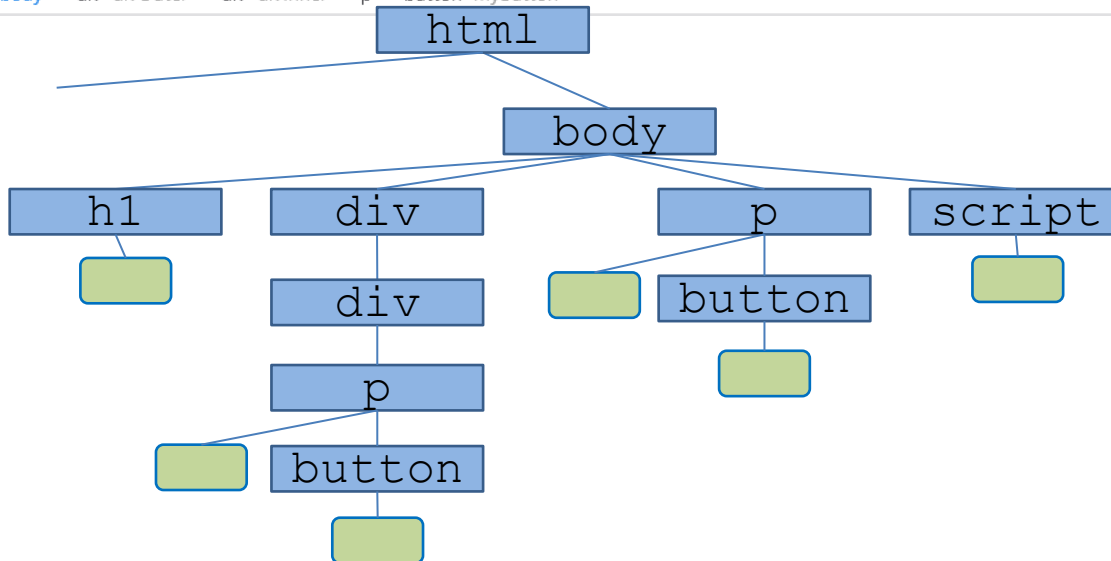


The screenshot shows a web development tool interface. At the top, there's a toolbar with icons for Inspektor, Konsola, Debugger, Sieć, Edytor stylów, Wydajność, Pamięć, and Dane. Below the toolbar is a search bar labeled "Szukaj w kodzie HTML". The main area displays the HTML code for the body element, which is expanded to show the following structure:

```
<!--Nie uruchamiać w Live server'ze, nie zrenderuje HTML przed poniższym skryptem-->
<h1>Event traversal</h1>
<div id="divOuter">
  <div id="divInner">
    <p>
      Press the button
      <button id="myButton" type="button">Press</button>
    </p>
  </div>
</div>
<p>
  Button for reset:
  <button id="myReset" type="button">Reset</button>
</p>
<p id="info"></p>
<script></script>
```

The breadcrumb at the bottom reads: `html > body > div#divOuter > div#divInner > p > button#myButton`.

Some browsers create text elements with blank text, e.g. for the above layout between `<button>` a `<p>`



The screenshot shows the KONSPEKT panel in VS Code, which displays the DOM tree structure. The tree is as follows:

- html
 - head
 - meta
 - meta
 - meta
 - title
 - link
 - script
 - body
 - h1
 - div#divOuter
 - div#divInner
 - p
 - button#myButton
 - p
 - button#myReset
 - p#info
 - script
 - oldColor
 - addEventListener("click") callback
 - addEventListener("click") callback
 - addEventListener("click") callback
 - addEventListener("click") callback

DOM - possibilities

- With the object model, JavaScript gets all the power it needs to create dynamic HTML:
 - JavaScript can change all HTML **elements** in a page
 - JavaScript can change all HTML **attributes** in a page
 - JavaScript can change all CSS **styles** on a page
 - JavaScript can **remove** existing HTML **elements** and **attributes**
 - JavaScript can **add** new HTML **elements** and **attributes**
 - JavaScript can **respond** to all existing HTML **events** on a page
 - JavaScript can create **new** HTML **events** on a page
- DOM is a standard object model and programming interface for HTML. Specifies:
 - HTML elements as objects
 - Properties of all HTML elements
 - Methods for accessing all HTML elements
 - Events for all HTML elements
 - An HTML DOM is a standard for retrieving, changing, adding, or removing HTML elements.
 - HTML DOM methods are actions that you can perform (on HTML elements).
 - HTML DOM properties are values (HTML elements) that you can set or change.

DOM – find items

- In addition to the method for retrieving references to an object with a specified identifier, you can retrieve **collections of items**:
 - With the same **tag name**:
`getElementsByTagName (tagName)`
 - With the same **class name**:
`getElementsByClassName (className)`
 - Based on a **CSS selector** that returns **the first element** that meets a rule: `querySelector (selector)`
 - Based on a **CSS selector** that returns **all items** that match the rule: `querySelectorAll (selector)`
- The above methods are often applied to the `document` object, but they can also be applied to **any DOM element**.
 - So the methods then work for the selected DOM **subtree**, for all elements "**under**" **the selected one**

Attributes of HTML elements

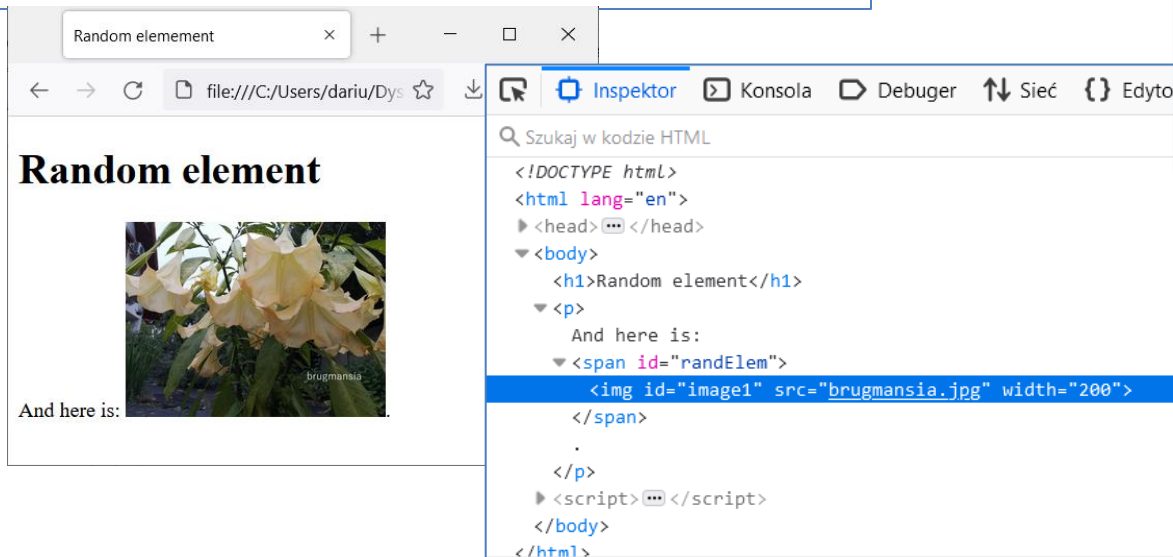
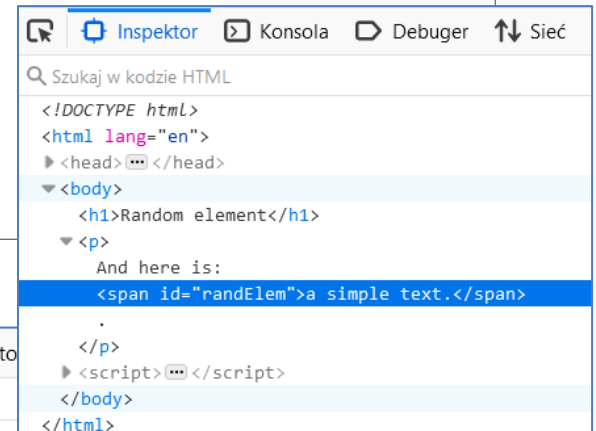
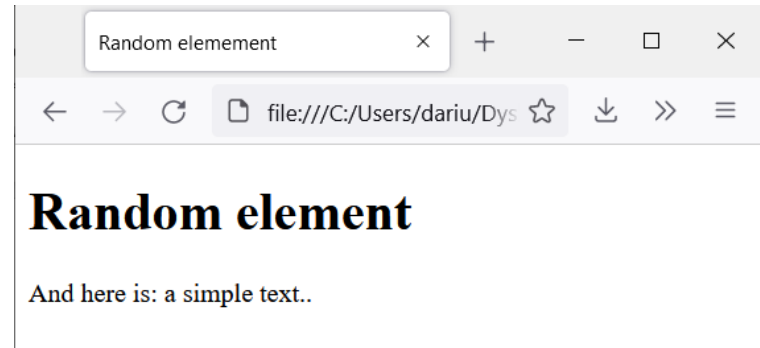
- You can also manipulate element attributes by using methods:
 - `getAttribute(attributeName)`
 - `setAttribute(attributeName, attributeValue)`
- Attributes are also available through the **dot operator** and **attribute name**
 - If there are **minuses** in the attribute name (**kebab-case** notation) it is replaced with **camel notation** (**camelCase**)
 - For example, the attribute `background-color` will be converted to a field `backgroundColor`.
 - Certain attributes that are **reserved words** have a different name, e.g..
`className`
 - There are **not** `elem.class`, but you can also use `elem.setAttribute("class", addClassStr);`
 - Or `elem["class"]=setClassStr;`
 - There is also a `propertyclassList` with methods `add()`, `contains()`, `remove()` etc.
- Similarly, the style properties that are in the `style` field behave in a similar way. Use *`element.style.property`*:
 - For example, if you want to get to the background color saved in the style of the element `elem` you should use `elem.style.backgroudColor`.
- This is equivalent to inline styling.
- **Note:** An **item must be rendered** to be assigned the correct style from the CSS rules.
 - For example, a newly created element by `document.createElement(elementName)` has default styles.

Example – randomly image or text

- Example: A randomly selected element in an element `<span id="randElem">`.
 - Either an image or plain text.
 - Different ways to set attributes for an image.
 - Using the `Math` object and its static functions:
 - `random()` – returns a random value in the interval `<0;1)`
 - `floor(x)` – returning the highest integer not greater than `x`.
- File: `randomElem.html`

Execution – random image or text

```
<body>
  <h1>Random element</h1>
  <p>
    And here is: <span id="randElem"></span>.
  </p>
  <script>
    if(Math.floor((Math.random()*2))==0){
      var elem=document.createElement("img");
      elem.src="brugmansia.jpg";
      elem.setAttribute("width","200");
      elem["id"]="image1";
    }
    else{
      var elem=document.createTextNode(" a simple text.");
    }
    var parent=document.getElementById("randElem");
    parent.appendChild(elem);
    // document.getElementById("randElem").appendChild(elem);
  </script>
</body>
```



Examples - Font style changes

- Example: dom1.html
 - The font of paragraph #p1 turns red when you click the button that says "Press!,"
 - `onclick` attribute – event handling (details on later slides)
 - The font of paragraph #p2 changes its characteristics after the script is finished.

```
<body>
<h2>JavaScript HTML DOM</h2>
<p>Changing the HTML style:</p>
<p id="p1">Hello World!</p>
<p id="p2">Hello World!</p>
<button type="button" onclick="document.getElementById('p1').style.color = 'red'"> Press!</button>
<script>
  document.getElementById("p2").style.color = "blue";
  document.getElementById("p2").style.fontFamily = "Arial";
  document.getElementById("p2").style.fontSize = "larger";
</script>
</body>
</html>
```

JavaScript HTML DOM

Changing the HTML style:

Hello World!

Hello World!

Press!

JavaScript HTML DOM

Changing the HTML style:

Hello World!

Hello World!

Press!

Selected operations in the DOM tree

For an element `elem`:

- **Add an item to the end of your children list:**
 - `elem.appendChild(childNode)`
- **Add an item on the begin of the children list:**
 - `elem.prepend(childNode)`
- **Remove a selected child from a parent element:**
 - `elem.removeChild(childNode)`
- **Access to your first or last child:**
 - `elem.firstChild`
 - `elem.lastChild`
- **Child replacement at the parent (returns the replaced item):**
 - `elem.replaceChild(newChild, oldChild)`
- **Parent access:**
 - `elem.parentNode`
- **Access to all children:**
 - `elem.childNodes`
 - **Loop for all children:** `for (var child of node.childNodes)`

Example domOperations.html/.css- operations on DOM elements

```
<h1>DOM operations</h1>
<div id="p1">
  <article id="art3"> Article3</article>
  <article id="art1"> Article1</article>
</div>
<h1 id="myH1"> Next part </h1>
<p>
  
  Lorem ipsum (...).
</p>
<script>
  var p1=document.getElementById("p1");
  var art1=document.getElementById("art1");
  p1.removeChild(art1);
  p1.prepend(art1);
  var art2=document.createElement("article");
  art2.id="art2";
  art2.innerText="Article2";
  p1.insertBefore(art2,document.getElementById("art3"));
  p1.parentNode.style.backgroundColor="yellow";
  p1.appendChild(document.getElementById("img1"));
</script>
</body>
```

```
article {
  background-color: aqua;
  border: brown 2px solid;
  margin: 3px 3px 3px 20px;
}

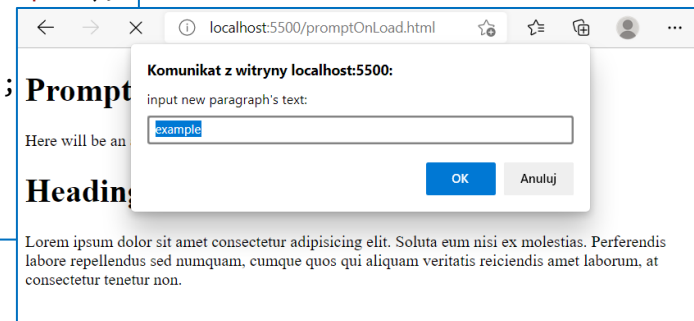
#art2 {
  font-size: 200%;
}
```



Events on a web page

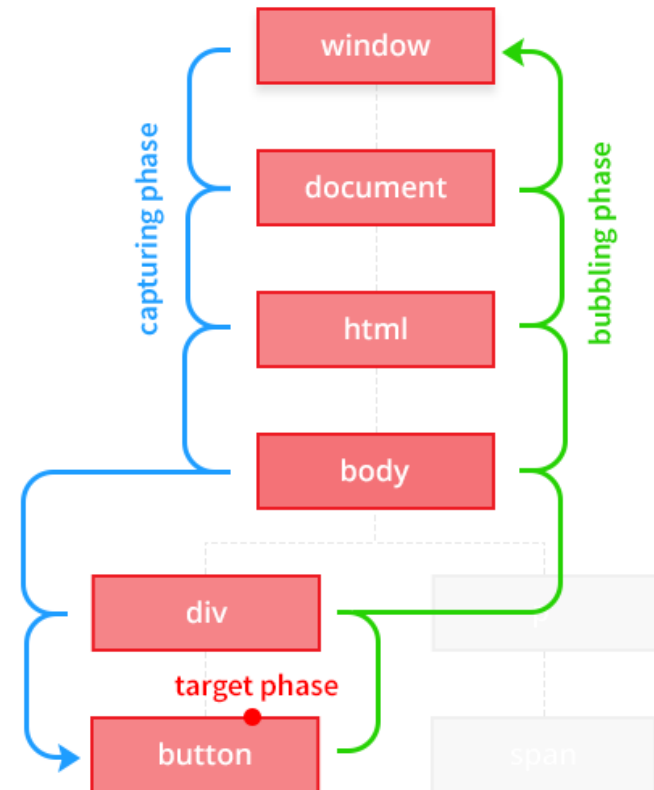
- In order for the page to load first **and then execute** the script, you need to make it a function that will start at the time of the `load` event for the `window` object (indicating that the page **has been loaded and rendered**).
- This involves, for example, registering a function in the listener of the event.
 - There can be many functions added to the event listener.
 - You can also remove a feature from an event listener.
- Examples of possible events: „load“, „click“, „focus“ etc.
- Method
`window.addEventListener(eventName, functionName, duringCapture)`
adds window's listener (window) dla zdarzenia eventName functionName. Parameter duringCapture will be discussed further.
 - method `addEventListener()` exists **for all elements** of an HTML document (and not only).
- Example: `promptOnLoad.html`

```
<script>
  window.addEventListener("load",
    function(){
      var text1=window.prompt("input new paragraph's text:", "example");
      var pElem=document.getElementById("p1");
      pElem.innerHTML=text1; // tag rendering
      var text1=window.prompt("input new heading text:", "Main");
      var pElem=document.getElementById("myH1");
      pElem.innerText=text1; // no rendering
    }, false);
</script>
```



Event phases

- The event takes place in 3 phases:
 - **capture phase** - when an event goes down the tree (from `window`) to a given element
 - **target phase** - when the event reaches the element that triggered this event
 - **bubbling phase** - when the event climbs up the tree until it reaches the `window`
- The same event can be handled in many places of the DOM tree on the above-mentioned paths.
- When registering the handling method, in the third parameter, we inform, if the service is given to be during the phase capture (`true`) or during the phase bubbling (`false`, default).
 - For the target phase, it doesn't matter.
- Image source:
<https://kursjs.pl/kurs/events/events.php>



Event handler function

- The event handler function receives event `e` as a parameter, which contains, among other things,:
 - `e.type` – event type without prefix `on-`
 - `e.target` – target phase element
 - `e.currentTarget` – the element that currently handles the event
 - `e.key` – pressed key
 - `e.code` – key-down code (text)
 - `e.ctrlKey` – whether the key Ctrl has been pressed
 - `e.shiftKey` – whether the key Shift has been pressed
 - `e.altKey` – whether the key Alt has been pressed
 - `e.clientX` i.e. `e.clientY` – mouse coordinates relative to the user client area (browser viewport)
 - `e.screenX`, `e.screenY` – mouse coordinates relative to the entire screen of the device
- Test program:
 - Dependence: `#divOuter > #divInnder > p > #myButton`
 - Clicking the `#myButton` button causes the operation to be performed in the capture phase `<div id="divInner">`,
 - Change the background color of this item
 - Then the event arrives at the item `<button id="myButton">`
 - Write an alert „before“, will change the color of the button, write an alert „after“
 - In the element `<p id="info">` will write data on the event
 - To reach the element in the bubbling phase `<div id="divOuter">`
 - Changes the background color of this item
 - Pressing buton „Reset“ undoes all changes.
 - The button must be outside the item `#divOuter`, otherwise the event „click“ will start for this item.
- Files for example:
 - `eventTraversal.html`
 - `eventTraversal.css`
 - `eventInfo.js` – with a helper function `eventInfo(e)` that returns a string with information about the event `e`.

Example eventTraversal.html/.css/.js - codes

```
<body>
  <!-- Dont run with Live server -->
  <h1>Event traversal</h1>
  <div id="divOuter">
    <div id="divInner">
      <p>
        Press the button <button type="button" id="myButton">Press</button>
      </p>
    </div>
  </div>
  <p>
    Button for reset:<button type="button" id="myReset">Reset</button>
  </p>
  <p id="info"></p>
</script>
var oldColor=document.getElementById("myButton").style.backgroundColor;
document.getElementById("myButton").addEventListener("click",
  function(e){
    window.alert("before"); // step 2
    e.target.style.backgroundColor="red"; //step 3
    document.getElementById("info").innerHTML=eventInfo(e);
    window.alert("after"); // step 4
  },false); // target phase
document.getElementById("divOuter").addEventListener("click",
  function(e){
    //window.alert("jestem");
    e.currentTarget.style.backgroundColor="green";
  },false); // bubbling phase // step 5
document.getElementById("divInner").addEventListener("click",
  (e)=>{
    e.currentTarget.style.backgroundColor="aqua";
  },true); // capture phase (step 1)
document.getElementById("myReset").addEventListener("click",
  ()=>{
    document.getElementById("myButton").style.backgroundColor=oldColor;
    document.getElementById("divOuter").style.backgroundColor="yellow";
    document.getElementById("divInner").style.backgroundColor="pink";
    document.getElementById("info").innerHTML="";
  },false); // target phase
</script>
</body>
```

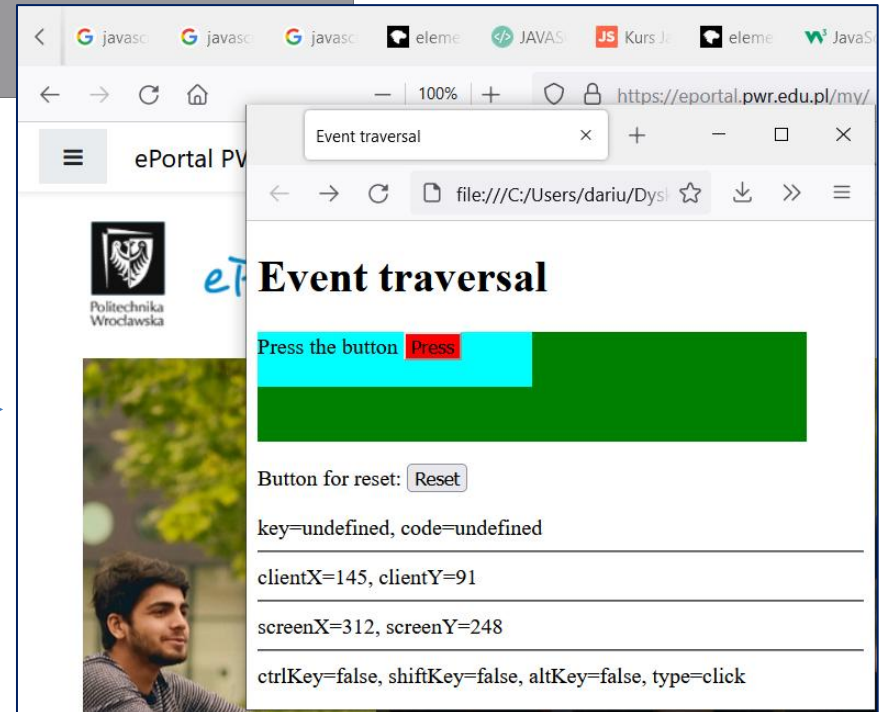
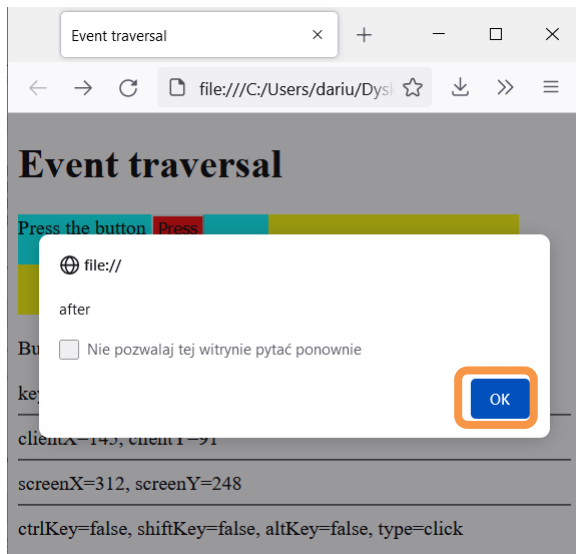
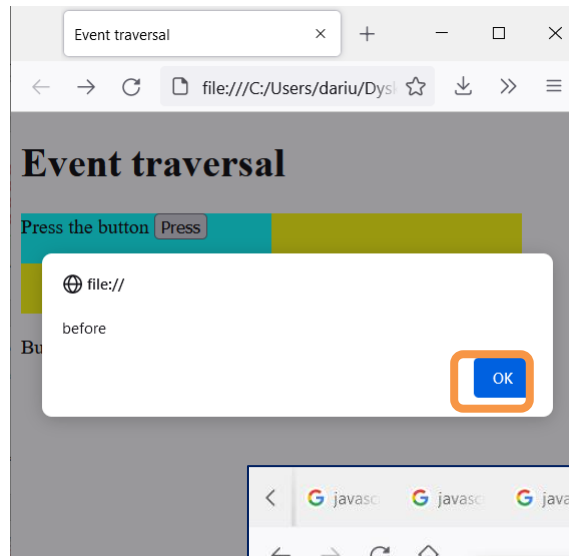
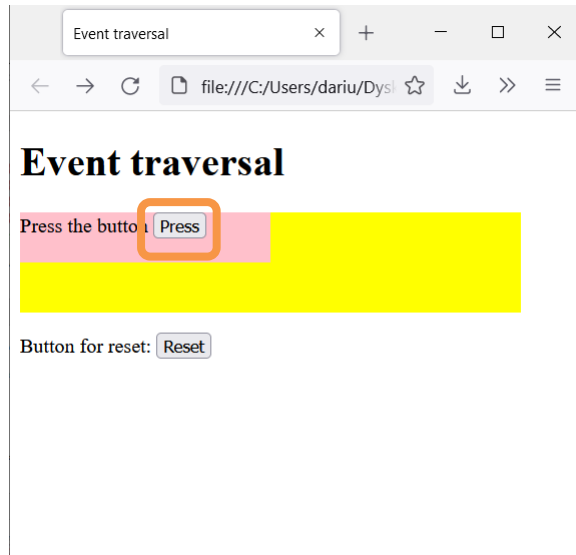
```
#divOuter{
  width: 400px;
  height: 80px;
  background-color: yellow;
}

#divInner{
  width: 200px;
  height: 40px;
  background-color: pink;
}
```

```
function eventInfo(e){
  return "key="+e.key+
    ", code="+e.code+" <hr> "+
    "clientX="+e.clientX+
    ", clientY="+e.clientY+" <hr> "+
    "screenX="+e.screenX+
    ", screenY="+e.screenY+" <hr> "+
    "ctrlKey="+e.ctrlKey+
    ", shiftKey="+e.shiftKey+
    ", altKey="+e.altKey+
    ", type="+e.type;
}
```

Note: The behavior of some browsers differs from that shown on the slides, because they postpone the reendering operation until the JS script is finished (no intermediate states of the web page appear).
The final look, of course, is always the same.

Example - execution



Events – programming tool

```
<head>...</head>
<body>
  <!--Nie uruchamiać w Live server'ze, nie zrenderuje HTML przed poniższym skryptem-->
  <h1>Event traversal</h1>
  <div id="divOuter"> event
    <div id="divInner"> ent
      <p>
        Press the button
        <button id="myButton" type="button">Press</button> event
      </p>
    </div>
  </div>
```

```
<body>
  <!--Nie uruchamiać w Live server'ze, nie zrenderuje HTML przed poniższym skryptem-->
  <h1>Event traversal</h1>
  <div id="divOuter"> event
    <div id="divInner"> event
      <p>
        Press the but ▶ click ...NET/Wyk%C5%82ad/W04/eventTraversal.html:40:12 ↗ Przechwytywanie DOM2
        <button id="myButton" type="button">Press</button> event
      </p>
    </div>
  </div>
```

```
<body>
  <!--Nie uruchamiać w Live server'ze, nie zrenderuje HTML przed poniższym skryptem-->
  <h1>Event traversal</h1>
  <div id="divOuter"> event
    <div id="divInner"> event
      <p>
        Press the button
        <button id="myButton" type="button">Press</button> event
      </p>
    </div>
  </div>
```

Other selected HTML DOM events

- Event:
 - General: `change`, `invalid`, ...
 - In the browser: `resize`, `scroll`, ...
 - Keyboard: `keypress`, `keydown`, `keyup`, `focusin`, `focus`, `focusout`, `blur`, `copy`, `paste`...
 - Mouse: `mouseover`, `mousedown`, `mouseenter`, `mouseleave`, ...
 - Other: `beforeprint`, `submit`, `clear`, ...
- Example (`keyEvent.html`):
 - A text box that shows information about an event in the element `p#myInfo`.
 - Used function `eventInfo(e)` from the file `eventTraversal.js`.
 - You want the field to turn **red** when the number of characters in the field is **divisible by 3**. If not, then the change of color to "cyan".
 - Test pressing Alt, Ctrl, Shift in conjunction with printable character keys

Example

```
<body>
  <h1>Key events</h1>
  <p id="p1">
    <label>Input: <input id="myInput" type="text"
      name="name" onkeyup="count3(event)"></label>
  </p>
  <p id="info"></p>
  <script>
    function count3(e){
      var count=e.target.value.length;
      document.getElementById("info").innerHTML=eventInfo(e);
      var styler=document.getElementById("myInput").style;
      if(count%3==0)
        styler.backgroundColor="red";
      else
        styler.backgroundColor="cyan";
    }
  </script>
</body>
```

Key events

Input: xyzW

key=., code=Semicolon

clientX=undefined, clientY=undefined

screenX=undefined, screenY=undefined

ctrlKey=true, shiftKey=true, altKey=true type=keyup

Key events

Input:

Key events

Input: x

key=x, code=KeyX

clientX=undefined, clientY=undefined

screenX=undefined, screenY=undefined

ctrlKey=false, shiftKey=false, altKey=false, type=keyup

Key events

Input: xyz

key=z, code=KeyZ

clientX=undefined, clientY=undefined

screenX=undefined, screenY=undefined

ctrlKey=false, shiftKey=false, altKey=false, type=keyup

Key events

Input: xyzW

key=W, code=KeyW

clientX=undefined, clientY=undefined

screenX=undefined, screenY=undefined

ctrlKey=false, shiftKey=true, altKey=false, type=keyup

Attach event handlers - Attributes with prefix `on` -

- Another possibility (old) is to assign a JS function to an element attribute. Attribute name is the prefix `on` followed by the event name
- You insert an attribute with the prefix `on` and the name of the event (e.g. `onload`, `onclick`, `onfocus`, etc.), and the value is given as JavaScript code (most often - method call).
 - E.a.: `<body onload="start()">`
- This **unregisters all** previously registered features.
- **Only one** method can be added this way.

Drag and drop events

- The `ondrag` event occurs when you drag an item or select text.
- Drag and drop is a very common feature in HTML5. This happens when you "grab" an object and drag it to another place.
- To allow you to drag an element, use the attribute `draggable=true`.
 - Note: Links and images can be dragged by default and do not require a `draggable` attribute.
- There are many events that are used and can occur at different stages of drag-and-drop operations.
- Events running on a dragging element (source element):
 - `ondragstart` - occurs when the user starts dragging an item
 - `ondrag` - occurs when an item is dragged
 - `ondragend` - occurs when the user finishes dragging the item
- Events running on a target of dragging:
 - `ondragenter` - occurs when a dragging element enters the drop target
 - `ondragover` - occurs when the element you are dragging is above the drop target
 - `ondragleave` - occurs when a dragged element leaves the drop target
 - `ondrop` - occurs when a dragging element is dropped on a drop target
- **Note:** When you drag an item, the `ondrag` event runs **every 350 milliseconds**.

Example - drag & drop – code 1/2

- Example: drag.html

```
<title>Drag & drop</title>
<style>
.droptarget {
  float: left;
  width: 100px;
  height: 35px;
  margin: 15px;
  padding: 10px;
  border: 2px solid #aaa;
}
</style>
</head>
<body>
```

```
<p>Move the p element between the following rectangles:</p>
<div style="background-color: lightgreen;" class="droptarget" id = "1"
ondrop="drop(event)" ondragover="allowDrop(event)">
  <p style="background-color: red;" ondragstart="dragStart(event)"
    ondrag="dragging(event)" draggable="true" id="dragtarget">Move me!</p>
</div>

<div style="background-color: lightblue;" class="droptarget" id = "2"
ondrop="drop(event)" ondragover="allowDrop(event)"></div>

<div style="background-color: lightpink;" class="droptarget" id = "3"
ondrop="drop(event)" ondragover="allowDrop(event)"></div>

<div style="background-color: lightcoral;" class="droptarget" id = "4"
ondrop="drop(event)" ondragover="allowDrop(event)"></div>

<div style="background-color: lightskyblue;" class="droptarget" id = "5"
ondrop="drop(event)" ondragover="allowDrop(event)"></div>

<div style="background-color: greenyellow;" class="droptarget" id = "6"
ondrop="drop(event)" ondragover="allowDrop(event)"></div>
<div><p id="demo"></p></div>
```

Example - drag & drop – code 2/2

```
<script>
function dragStart(event) {
    event.dataTransfer.setData("Text", event.target.id);
    document.bgColor="white";
}
function dragging(event) {
    document.getElementById("demo").innerHTML = "Element &lt;p&gt; Move me! &lt;p&gt; has been raised";
}
function allowDrop(event) {
    event.preventDefault();
}
function drop(event) {
    event.preventDefault();
    var data = event.dataTransfer.getData("Text");
    event.target.appendChild(document.getElementById(data));
    var str="Element &lt;p&gt; Move me! &lt;p&gt; has been dropped"+" and is in the field "+event.target.id
    document.getElementById("demo").innerHTML = str;
    document.bgColor=document.getElementById(event.target.id).style.backgroundColor;
    // the entire document will take over the background color of the element from the event - item dropping
}
</script>

</body>
```

Example - drag & drop - execution

Move the p element between the following rectangles:



Move the p element between the following rectangles:



Element <p> Move me! <p> has been raised

Move the p element between the following rectangles:



Element <p> Move me! <p> has been dropped and is in the field 5

Other JS elements

- Exception mechanism – block `try/catch`
- Objects Iterable
 - They have a method `next()`
- Collections `Map`, `Set`, `Array`.
- Loops:
 - `for (key in object) { ... }`
 - `for (variable of iterable) {...} // (ES6 2015)`
- **General note:** It is worth checking if the selected solution (function in JavaScript) is not too slow and use other library functions.
 - Scripting languages often have a time mark-up associated with continuous code compilation, so in addition to an effective algorithm, you should choose an effective way to implement it.
- jQuery – JavaScript library for easy operation on the DOM model
 - Uses CSS selectors and enters a `$` object e.a.:
`$("button.continue").html("Next Step...")`
- ...